



**T.C.
NECMETTİN ERBAKAN
ÜNİVERSİTESİ
SEYDİŞEHİR AHMET CENGİZ
MÜHENDİSLİK FAKÜTESİ**



BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

**MİKROSERVİS MİMARİSİ KULLANILARAK ÖLÇEKLENEBİLİR BİR E-
TİCARET PLATFORMUNUN TASARIMI VE DOCKER İLE KONTEYNER
TABANLI DAĞITIMI**

22370031033-Furkan KILINÇ

**Danışman:
Dr. Öğr. Üyesi Yunus Emre Göktepe**

MÜHENDİSLİK PROJESİ I

**Kasım-2025
SEYDİŞEHİR
Her Hakkı Saklıdır**

LİSANS BİTİRME ÖDEVİ SONUÇ FORMU

..... tarafından
danışmanlığında hazırlanan “.....” başlıklı lisans
bitirme ödevi tarafımızdan incelenmiş, kapsamı ve niteliği açısından, ... / ... / 20...
tarihinde bir Lisans Bitirme Ödevi olarak kabul edilmiştir / edilmemiştir.

Unvan Ad SOYAD

Danışman

Unvan Ad SOYAD

Jüri Üyesi

Unvan Ad SOYAD

Jüri Üyesi

Unvan Ad SOYAD

Bölüm Başkanı

TEZ BİLDİRİMİ

Bu tezdeki bütün bilgilerin etik davranış ve akademik kurallar çerçevesinde elde edildiğini ve tez yazım kurallarına uygun olarak hazırlanan bu çalışmada bana ait olmayan her türlü ifade ve bilginin kaynağına eksiksiz atıf yapıldığını bildiririm.

DECLARATION PAGE

I hereby declare that all information in this document has been obtained and presented in accordance with academic rules and ethical conduct. I also declare that, as required by these rules and conduct, I have fully cited and referenced all material and results that are not original to this work.

Seydişehir,/...../ 20...

İmza

Furkan KILINÇ

ÖZET

MİKROSERVİS MİMARİSİ KULLANILARAK ÖLÇEKLENEBİLİR BİR E-TİCARET PLATFORMUNUN TASARIMI VE DOCKER İLE KONTEYNER TABANLI DAĞITIMI

Bu tez çalışması, günümüzün hızla dijitalleşen ticaret dünyasında artan ölçeklenebilirlik, esneklik ve sürekli kullanılabilirlik gereksinimlerine çözüm sunmayı hedeflemektedir. Geleneksel monolitik yazılım mimarilerinin, büyük ve karmaşık e-ticaret sistemlerinde karşılaşılan bakım zorlukları, yavaş geliştirme süreçleri ve sınırlı ölçeklenebilirlik gibi problemler karşısında yetersiz kaldığı görülmektedir. Bu çalışma, söz konusu sorunları aşmak amacıyla modern bir yazılım yaklaşımı olan mikroservis mimarisini temel almaktadır.

Tezin özgün değeri, bir e-ticaret platformunun temel işlevlerini (Ürün Kataloğu, Kullanıcı Yönetimi, Sipariş Yönetimi ve Ödeme İşlemleri gibi) bağımsız, otonom ve kendi veritabanına sahip mikroservislere ayırarak, monolitik mimarilere kıyasla sağlanan avantajları uygulamalı olarak ortaya koymasındır. Her bir mikroservisin bağımsız şekilde geliştirilebilmesi, dağıtılabilmesi ve ölçeklenebilmesi sayesinde sistemin esnekliği ve sürdürülebilirliği artırılmaktadır.

Çalışmanın yönteminde, mikroservis mimarisinin etkin bir şekilde uygulanabilmesi için Docker konteyner teknolojisinden faydalanılmıştır. Her bir mikroservis, Docker imajları ile paketlenerek ortamdaki bağımsız hale getirilmiş; Docker Compose kullanılarak tüm sistemin kolayca ayağa kaldırılması, test edilmesi ve yönetilmesi sağlanmıştır. Bu yaklaşım, geliştirme ve dağıtım süreçlerinde tutarlılık sağlarken, sistemin farklı ortamlarda sorunsuz çalışmasına olanak tanımaktadır.

Servisler arası iletişim sürecinde, dağıtık sistemlerin doğasından kaynaklanan karmaşıklıkları yönetebilmek amacıyla hem senkron (RESTful API) hem de asenkron (mesaj kuyruğu tabanlı) iletişim yöntemleri tasarlanmıştır. Bu sayede, servisler arasındaki bağımlılıkların azaltılması, hata izolasyonunun artırılması ve sistem genelinde daha dayanıklı bir yapı oluşturulması hedeflenmiştir.

Tez kapsamında, tasarlanan mikroservis mimarisinin performansı, ölçeklenebilirliği ve hata toleransı değerlendirilmiş; elde edilen bulgular, mikroservis ve Docker entegrasyonunun monolitik yapılarla kıyasla önemli avantajlar sunduğunu göstermiştir. Özellikle hızlı dağıtım, seçici ölçeklendirme ve servis bazlı hata yönetimi konularında mikroservis mimarisinin etkinliği somut bir e-ticaret uygulaması üzerinden ortaya konmuştur.

Bu çalışma, modern yazılım mimarileri ve konteyner teknolojilerinin e-ticaret sistemlerindeki uygulanabilirliğini göstererek hem akademik literatüre hem de sektörel uygulamalara katkı sağlamayı amaçlamaktadır. Sonuç olarak, mikroservis mimarisi ve Docker tabanlı konteynerleşme yaklaşımı, yüksek trafikli ve sürekli gelişim gerektiren e-ticaret platformları için sürdürülebilir, esnek ve yüksek performanslı bir çözüm sunmaktadır.

Anahtar Kelimeler: Docker, E-ticaret, Konteyner, Mikroservis, Ölçeklenebilirlik, Dağıtık Sistemler.

ABSTRACT

DESIGN OF A SCALABLE E-COMMERCE PLATFORM USING MICROSERVICES ARCHITECTURE AND CONTAINER-BASED DEPLOYMENT WITH DOCKER

This thesis aims to address the increasing demands for scalability, flexibility, and continuous availability in today's rapidly digitalizing commercial environment. Traditional monolithic software architectures are observed to be insufficient in the face of maintenance challenges, slow development processes, and limited scalability encountered in large and complex e-commerce systems. To overcome these challenges, this study is based on the microservices architecture as a modern software development approach.

The original contribution of this thesis lies in practically demonstrating the advantages of microservices architecture over monolithic architectures by decomposing the core functionalities of an e-commerce platform—such as Product Catalog, User Management, Order Management, and Payment Processing—into independent, autonomous microservices, each with its own dedicated database. The ability of each microservice to be developed, deployed, and scaled independently enhances the system's flexibility and sustainability.

In the methodology of this study, Docker container technology is utilized to enable the effective implementation of microservices architecture. Each microservice is packaged as a Docker image, ensuring environment independence, while Docker Compose is employed to facilitate the deployment, testing, and management of the entire system. This approach ensures consistency across development and deployment processes and enables seamless operation in different environments.

To manage the inherent complexity of distributed systems, both synchronous (RESTful API) and asynchronous (message queue-based) communication methods are designed for inter-service communication. This design aims to reduce inter-service dependencies, enhance fault isolation, and build a more resilient overall system architecture.

Within the scope of the thesis, the performance, scalability, and fault tolerance of the designed microservices architecture are evaluated. The results demonstrate that the integration of microservices and Docker offers significant advantages over monolithic architectures, particularly in terms of rapid deployment, selective scaling, and service-level fault management. The effectiveness of the microservices approach is concretely demonstrated through a practical e-commerce application.

This study aims to contribute to both academic literature and industry practices by demonstrating the applicability of modern software architectures and container technologies in e-commerce systems. In conclusion, the microservices architecture and Docker-based containerization approach provide a sustainable, flexible, and high-performance solution for high-traffic e-commerce platforms that require continuous evolution.

Keywords: Docker, E-commerce, Container, Microservices, Scalability, Distributed Systems.

ÖNSÖZ

Bu bitirme projesi kapsamında, modern yazılım mimarilerinin en güncel yaklaşımlarından biri olan Mikroservis Mimarisi kullanılarak; ölçeklenebilir, esnek ve yüksek performanslı bir e-ticaret platformu geliştirilmektedir. Projenin temel amacı, geleneksel monolitik yapıların getirdiği kısıtlamaları aşarak, konteynerizasyon (Docker) ve servisler arası bağımsız veri yönetimi (Polyglot) prensipleriyle sürdürülebilir bir sistem tasarımı ortaya koymaktır.

Çalışma süresince .NET, Docker, PostgreSQL, Portainer , Homer , Seq ve Redis gibi teknolojiler bir arada kullanılarak dağıtık sistemlerin karmaşıklığına yönelik pratik çözümler üretilmiştir. Bu süreçte sadece teknik bir uygulama geliştirmekle kalmayıp, modern yazılım mühendisliği standartları üzerine derinlemesine bir deneyim kazanılmıştır.

Öncelikle, bana bu araştırmayı yapma fırsatı sunan Necmettin Erbakan Üniversitesi'ne şükranlarımı sunarım. Proje süresince değerli bilgilerini, vizyonunu ve desteğini esirgemeyen, her tıkandığımda yol gösteren değerli danışman hocam Dr. Öğr. Üyesi Yunus Emre Göktepe'ye en içten teşekkürlerimi sunarım.

Ayrıca, hayatım boyunca olduğu gibi bu zorlu akademik süreçte de sabırları, maddi ve manevi destekleriyle daima yanımda olan, bana güvenen sevgili aileme sonsuz minnettarım. Bu projenin, mikroservis ve konteyner teknolojilerine ilgi duyan diğer öğrenci arkadaşlarıma ve akademik literatüre faydalı bir kaynak olmasını temenni ederim.

22370031033- Furkan KILINÇ
SEYDİŞEHİR-2025

İÇİNDEKİLER

ÖZET	3
ABSTRACT.....	4
ÖNSÖZ	5
İÇİNDEKİLER	6
ŞEKİLLER LİSTESİ	7
ÇİZELGELER/TABLolar LİSTESİ.....	8
SİMGELER VE KISALTMALAR.....	9
1. GİRİŞ.....	10
2. KAYNAK ARAŞTIRMASI	13
2.1. Servis Tabanlı Platformların Karşılaştırılması	14
2.2. API Gateway.....	14
2.3. Redis ve Dağıtık Önbellekleme	15
2.4. Event-Driven (Olay Güdümlü) Mimari	15
2.5. Bağımsız Veritabanları ve Poliglot Kalıcılık	16
2.6. Dağıtık Arama Sistemleri ve Log Yönetimi	16
2.7. Micro-Frontend Mimarisi	17
2.8. Konteynerizasyon ve Orkestrasyon (Kubernetes)	17
2.9. Hizmet Ağı Mimarisi	18
2.10. Sürekli Entegrasyon ve Sürekli Dağıtım (CI/CD)	18
2.11. Kimlik Doğrulama ve Yetkilendirme (OAuth2 ve JWT)	18
2.12. Altyapı Olarak Kod (Infrastructure as Code - IaC)	18
2.13. Yazılım Test Piramidi ve Test Otomasyonu.....	19
2.14. Sunucusuz (Serverless) Hesaplama Modeli.....	19
2.15. Gözlemlenebilirlik (Observability) ve Dağıtık İzleme	19
3. MATERYAL VE YÖNTEM.....	19
3.1. Materyaller.....	20
3.1.1. Kullanılan teknoloji ve kütüphaneler.....	20
3.1.2. Yazılım ve geliştirme araçları.....	21
3.1.3. Veritabanı ve veri yönetim sistemleri	22
3.2. Yöntem ve Mimari Tasarım.....	23
3.2.1. Mikroservis mimarisi yaklaşımı	23
3.2.2. Etki alanı odaklı tasarım	25
3.2.3. Servis içi mantıksal katmanlama	26
3.3. Veri Yönetimi ve İletişim	26
3.3.1. Servis başına veritabanı yaklaşımı.....	27
3.3.2. Servisler arası iletişim ve güvenlik (JWT)	28
3.4. Konteynerizasyon (Docker).....	30
3.4.1. Docker stratejisi ve Çok aşamalı derleme.....	30

3.4.2. Konteyner orkestrasyonu (Docker Compose).....	31
3.5. Merkezi İzleme ve Erişim Paneli.....	32
3.5.1. Docker stratejisi ve Çok aşamalı derleme.....	33
4. ARAŞTIRMA SONUÇLARI VE TARTIŞMA.....	33
4.1. Kapsam ve yaklaşım	34
4.2. Teknoloji yığını ve geliştirme ortamı	35
4.2.1. Ortam ve ön yüz.....	35
4.2.2. Arka uç ve altyapı	36
4.2.3. Mikroservis iskeletleri	37
4.3. Mimari ve veri modeli	37
4.3.2. Veritabanı varlıkları	38
4.3.3. Tutarlılık ve normalizasyon	38
4.4. Sunucu tarafı uygulama	39
4.4.1. HTTP sunucusu ve güvenlik orta katmanları	39
4.4.2. API modülleri	40
4.4.3. Kimlik doğrulama ve yetkilendirme	40
4.4.4. Sipariş oluşturma ve atomik işlem.....	41
4.4.5. Redis, koleksiyonlar ve kategori filtreleri.....	42
4.4.6. Dosya yükleme ve medya	43
4.5. İstemci uygulamaları.....	43
4.5.1. Mağaza.....	44
4.5.2. Yönetici paneli.....	44
4.5.3. Satıcı paneli.....	45
4.5.4. Kullanıcı deneyimi notları	45
4.6. İş kuralları ve senaryolar.....	46
4.6.1. Sepet.....	46
4.6.2. Çok satıcı pazar yeri	47
4.6. İş kuralları ve senaryolar.....	48
4.7. Güvenlik, günlük ve hata yönetimi	48
4.8. Monorepo, kod yapısı ve mimari perspektif.....	49
4.9. Performans, ölçek, risk ve test	50
4.10. Derleme, Docker ve dağıtım	50
4.10.1. Üretim derlemesi.....	51
4.10.2. Docker Compose.....	51
4.10.3. Ortam değişkenleri ve üretim güvenliği	52
4.11. REST ve örnek sözleşme	52
4.12. Sürüm kontrol	53
4.13. Bölüm özeti.....	53
5. SONUÇLAR VE ÖNERİLER.....	54
5.2 Öneriler	58
6. KAYNAKLAR	59

ŞEKİLLER LİSTESİ

Şekil 1.1. Bir harmonik titreşim hareketi	10
Şekil 1.2. Titreşim sistemi ve elemanları	15

ÇİZELGELER/TABLolar LİSTESİ

Çizelge 1.1. Bir harmonik titreşim hareketi	10
Çizelge 1.2. Titreşim sistemi ve elemanları	15

SİMGELER VE KISALTMALAR

KISALTMALAR

API	Application Programming Interface (Uygulama Programlama Arayüzü)
CI/CD	Continuous Integration / Continuous Deployment (Sürekli Entegrasyon / Sürekli Dağıtım)
DB	Database (Veritabanı)
DTO	Data Transfer Object (Veri Transfer Nesnesi)
JWT	JSON Web Token
MSA	Microservices Architecture (Mikroservis Mimarisi)
ORM	Object Relational Mapping (Nesne İlişkisel Eşleme)
REST	Representational State Transfer
RBAC	Role Based Access Control (Rol Tabanlı Erişim Kontrolü)
SQL	Structured Query Language (Yapılandırılmış Sorgu Dili)
UI	User Interface (Kullanıcı Arayüzü)
VM	Virtual Machine (Sanal Makine)

1. GİRİŞ

Günümüz e-ticaret sektörü, kullanıcı beklentilerinin hızla değiştiği, trafik yoğunluğunun dönemsel olarak büyük dalgalanmalar gösterdiği ve rekabetin sürekli yenilik gerektirdiği dinamik bir yapıya sahiptir. Bu ortamda, e-ticaret platformlarının yalnızca işlevsel olması yeterli olmamakta; aynı zamanda yüksek ölçeklenebilirlik, esneklik ve sürekli erişilebilirlik gibi özelliklere de sahip olması gerekmektedir. Ancak geleneksel monolitik yazılım mimarileri, büyük ve karmaşık sistemlerde bakım zorlukları, düşük geliştirme hızı ve sınırlı ölçeklenebilirlik gibi sorunlar nedeniyle bu gereksinimleri karşılamakta yetersiz kalmaktadır.

Monolitik yapılarda sistem bileşenlerinin birbirine sıkı şekilde bağlı olması, küçük bir güncelleme veya hata durumunda tüm uygulamanın etkilenmesine yol açmaktadır. Bu durum, hem sistemin tamamının tek bir hata noktasında devre dışı kalma riskini artırmakta hem de yeni özelliklerin sisteme entegrasyonunu zorlaştırmaktadır. Özellikle yüksek trafikli e-ticaret platformlarında bu tür kısıtlar, kullanıcı deneyimini olumsuz etkilemekte ve sistemin sürdürülebilirliğini tehdit etmektedir.

Bu sorunlara çözüm olarak geliştirilen Mikroservis Mimarisi (Microservices Architecture – MSA), uygulamanın bağımsız, otonom ve kendi yaşam döngüsüne sahip servisler hâlinde tasarlanmasını mümkün kılmaktadır. Mikroservis yaklaşımı sayesinde her bir işlevsel bileşen ayrı ayrı geliştirilebilmekte, dağıtılabilen ve ölçeklendirilebilmektedir. Bu yapı, sistemin genel dayanıklılığını artırırken, hata izolasyonu ve hızlı geliştirme süreçleri açısından önemli avantajlar sunmaktadır.

Araştırmanın Amacı

Bu araştırmanın amacı, yüksek trafikli ve sürekli değişen kullanıcı taleplerine sahip e-ticaret platformları için ölçeklenebilir, esnek ve dayanıklı bir yazılım altyapısı tasarlamaktır. Bu doğrultuda, platformun temel işlevleri (Ürün Kataloğu, Kullanıcı Yönetimi, Sipariş Yönetimi ve Ödeme İşlemleri) bağımsız, otonom ve kendi veritabanına sahip mikroservisler hâlinde yapılandırılmıştır. Mikroservis yaklaşımı, sistem bileşenlerinin bağımsız olarak geliştirilmesine, dağıtılmasına ve ölçeklenmesine imkân tanıırken, kod yönetimi, hata çözümlemesi ve hızlı geliştirme süreçleri açısından avantaj sağlamaktadır.

Araştırmanın Önemi

E-ticaret sistemleri, artan kullanıcı talepleri ve değişken trafik yoğunluğu nedeniyle esnek, ölçeklenebilir ve kesintisiz hizmet sunabilen altyapılara ihtiyaç duymaktadır. Geleneksel monolitik yaklaşımlar, bu gereksinimleri karşılamakta yetersiz kalmakta ve platformların büyüme potansiyelini ve hızını sınırlamaktadır. Bu çalışma, mikroservis mimarisi, konteyner teknolojileri, bağımsız veritabanları ve dağıtık sistem iletişim yöntemlerini bir arada kullanarak modern e-ticaret uygulamaları için uygulanabilir bir model sunmaktadır.

Projede her bir mikroservis, bağımsız olarak ölçeklenebilir ve yönetilebilir olacak şekilde Docker konteynerleri ile paketlenecek, Kubernetes ile orkestre edilerek performans ve kaynak kullanımı talebe göre optimize edilecektir. Sistem mimarisi, Frontend için React, Backend için Node.js, API Gateway için Nginx, mesaj aracısı olarak RabbitMQ ve her servisin kendi veritabanına sahip olacağı poliglot veri yönetimi yaklaşımına dayanmaktadır.

Bu bütünlük yaklaşım, hem akademik literatüre mikroservis ve konteyner tabanlı modern e-ticaret çözümlerinin uygulanabilirliğini göstermekte hem de sektörel uygulamalarda yüksek performanslı, esnek ve sürdürülebilir bir altyapı modeli sunmaktadır. Ayrıca, kullanıcı deneyimini kişiselleştirme ve veriye dayalı öneri sistemleri geliştirme açısından da önemli bir referans niteliği taşımaktadır.

Araştırma Yöntemleri

Bu çalışmada, e-ticaret platformu ölçeklenebilir mikroservis mimarisi kullanılarak tasarlanmıştır. Platformun temel servisleri arasında Kullanıcı Servisi, Ürün Servisi, Sepet Servisi, Sipariş Servisi, Ödeme Servisi, Bildirim Servisi, Yorum Servisi ve AI Öneri Servisi yer almaktadır. Her bir servis, bağımsız ve otonom olacak şekilde kendi veritabanına sahiptir ("DB per Service"), böylece veri izolasyonu ve poliglot veri yönetimi sağlanmaktadır. Servisler arası iletişim, hem senkron (RESTful API) hem de asenkron (RabbitMQ mesaj kuyrukları) yöntemleri ile sağlanarak bağımlılıkların azaltılması ve hata toleransının artırılması hedeflenmiştir.

Her bir mikroservis, Docker konteynerleri ile paketlenmiş ve Docker Compose kullanılarak geliştirme ve test ortamında kolayca ayağa kaldırılabilir hâle getirilmiştir. Üretim ortamında ise Kubernetes cluster'ı üzerinden orkestre edilerek, otomatik ölçeklendirme (Horizontal Pod Autoscaling), servis keşfi, yük dengeleme ve hata kurtarma gibi özelliklerle sistemin yüksek trafikte dahi dayanıklı ve performanslı

alışması saęlanmıřtır. Frontend React, Backend Node.js tabanlı olarak yapılandırılmıřtır. Tm servisler, merkezi bir API Gateway (Nginx) zerinden ynlendirilmekte, Elasticsearch ile veri ynetimi ve arama performansı artırılmakta, AI neri Servisi ise kullanıcıların davranıř verilerini analiz ederek kiřiselleřtirilmiř rn nerileri ve reklamlar sunmaktadır.

2. KAYNAK ARAŞTIRMASI

Bu bölümde, servis tabanlı ve mikroservis mimarileri kapsamında geliştirilen güncel yaklaşımlar incelenmiştir. Literatürde yer alan çalışmalar; mimari yapı (monolitik, SOA, mikroservis), altyapı bileşeni (önbellekleme, kalıcılık, orkestrasyon), iletişim modeli (senkron/asenkron) ve amaç (ölçeklenebilirlik, performans, kişiselleştirme) kriterlerine göre sınıflandırılmıştır.

Aşağıdaki alt başlıklarda, servis tabanlı mimarilerin evriminden başlayarak dağıtık önbellekleme, poliglot kalıcılık, API Gateway kullanımı, olay güdümlü mikroservisler, yapay zekâ destekli kişiselleştirme servisleri ve Kubernetes tabanlı organizasyon yaklaşımları ele alınmıştır.

2.1. Servis Tabanlı Platformların Karşılaştırılması

Servis tabanlı mimariler, yazılım sistemlerinin fonksiyonel olarak ayrıştırılmış ve bağımsız servisler şeklinde tasarlanmasını amaçlayan mimari yaklaşımlardır. Bu mimarilerin temel hedefi; yazılım sistemlerinde esneklik, sürdürülebilirlik ve ölçeklenebilirliğin artırılmasıdır. Servis Odaklı Mimari (Service-Oriented Architecture – SOA), bu yaklaşımın ilk sistematik örneklerinden biri olarak literatürde yer almaktadır. SOA, servislerin yeniden kullanılabilirliğini ve kurumsal sistemler arasında entegrasyonun kolaylaştırılmasını hedeflemiştir.

SOA mimarilerinde servisler arası iletişim çoğunlukla merkezi bir Enterprise Service Bus (ESB) üzerinden sağlanmakta ve SOAP tabanlı mesajlaşma ile WSDL gibi ağır protokoller kullanılmaktadır. Bu yapı, kurumsal ölçekte yönetilebilirlik sağlasa da zamanla performans sorunlarına ve mimari esneklik kaybına yol açmıştır. Literatürde ESB merkezli mimarilerin, yüksek gecikme süreleri ve değişime düşük adaptasyon kabiliyeti gibi dezavantajlara sahip olduğu ifade edilmektedir (Pautasso ve ark., 2008).

Bu sınırlamalar, daha hafif ve çevik bir mimari yaklaşım olan mikroservis mimarisinin ortaya çıkmasına zemin hazırlamıştır. Mikroservis mimarisi, uygulamaların küçük, bağımsız ve kendi yaşam döngüsüne sahip servisler şeklinde tasarlanmasını esas almaktadır. Yapılan çalışmalarda mikroservislerin bağımsız dağıtım, hata izolasyonu ve ölçeklenebilirlik açısından monolitik ve SOA tabanlı mimarilere kıyasla önemli avantajlar sunduğu vurgulanmaktadır (Dragoni ve ark., 2017).

2.2. API Gateway

Mikroservis mimarisinde istemcilerin çok sayıda servise doğrudan erişmesi, istemci tarafında karmaşıklığı artırmakta ve ağ trafiğinin yönetimini zorlaştırmaktadır. Bu problemi çözmek amacıyla literatürde API Gateway deseni önerilmektedir. API Gateway, istemciler ile mikroservisler arasında tek bir giriş noktası oluşturarak isteklerin merkezi biçimde yönetilmesini sağlar (Villamizar ve ark., 2016).

Akademik çalışmalarda API Gateway'in kimlik doğrulama, yetkilendirme, istek yönlendirme ve yanıt birleştirme gibi işlevleri tek bir katmanda topladığı belirtilmektedir. Bu yaklaşım, mikroservislerin yalnızca iş mantığına odaklanmasını sağlarken istemci-servis etkileşimini sadeleştirmektedir. Bununla birlikte, API Gateway katmanının tek hata noktası hâline gelmemesi için ölçeklenebilir ve yedekli tasarlanması gerektiği literatürde vurgulanmaktadır (Taibi ve ark., 2018).

2.3. Redis ve Dağıtık Önbellekleme

Dağıtık sistemlerde önbellekleme, sık erişilen verilerin hızlı bir şekilde sunulmasını sağlayarak sistem performansını artıran temel yaklaşımlardan biridir. Bellek içi veri depoları, veritabanı erişimlerini azaltarak yanıt sürelerini önemli ölçüde düşürmektedir. Literatürde anahtar-değer tabanlı bellek içi veri depolarının mikroservis mimarilerinde yaygın biçimde kullanıldığı belirtilmektedir (Grolinger ve ark., 2013).

Mikroservis tabanlı sistemlerde Redis gibi bellek içi anahtar-değer veri depolarının, önbellekleme amacıyla kullanılması performans ve ölçeklenebilirlik açısından önemli avantajlar sunmaktadır. Yapılan çalışmalarda Redis tabanlı önbellekleme yaklaşımlarının, yüksek trafikli sistemlerde düşük gecikme süresi sağladığı ve veritabanı üzerindeki yükü azalttığı ifade edilmektedir (Kanthed, 2023). Bununla birlikte, bellek içi veri depolarının genellikle kalıcı veri saklama gereksinimi olan sistemlerde yardımcı bir bileşen olarak konumlandırılması gerektiği literatürde vurgulanmaktadır.

2.4. Event-Driven (Olay Güdümlü) Mimari

Event-driven mimari, servisler arası iletişimin senkron çağrılar yerine asenkron olaylar üzerinden gerçekleştirilmesini esas almaktadır. Bu yaklaşımda servisler belirli olayları yayınlamakta ve diğer servisler bu olaylara abone olarak tepki vermektedir. Bu

sayede servisler arasında gevşek bağıllık sağlanmakta ve sistemin esnekliği artırılmaktadır (Hohpe ve Woolf, 2004).

Literatürde olay güdümlü mimarilerin yüksek ölçeklenebilirlik ve hata toleransı sunduğu belirtilmektedir. Ancak bu mimarinin, olayların izlenmesi, hata ayıklama ve sistem davranışlarının analiz edilmesi açısından daha karmaşık yapılar oluşturduğu ifade edilmektedir (Kleppmann ve ark., 2018).

2.5. Bağımsız Veritabanları ve Poliglot Kalıcılık

Mikroservis mimarisinin temel ilkelerinden biri, her servisin kendi veritabanına sahip olmasıdır. “Database per Service” olarak adlandırılan bu yaklaşım, servislerin veri katmanında da bağımsızlığını korumasını amaçlamaktadır. Bu sayede bir serviste yapılan veritabanı şeması veya teknoloji değişiklikleri, diğer servisleri doğrudan etkilememektedir (Pahl, 2015).

Geleneksel monolitik mimarilerde kullanılan merkezi veritabanı yaklaşımı, mikroservis mimarilerinde servisler arasında gizli bağımlılıklara neden olabilmektedir. Literatürde bu durumun, servislerin bağımsız ölçeklenmesini ve evrimini zorlaştırdığı belirtilmektedir (Dragoni ve ark., 2017). Bağımsız veritabanları yaklaşımı ise her servisin yalnızca kendi yüküne göre ölçeklenebilmesini mümkün kılarak sistem genelinde esneklik sağlamaktadır.

Bu yaklaşım, poliglot kalıcılık kavramının da yaygınlaşmasına katkı sağlamıştır. Poliglot kalıcılık, her mikroservisin kendi gereksinimlerine en uygun veritabanı teknolojisini kullanabilmesini ifade etmektedir. İlişkisel veritabanları tutarlılığın önemli olduğu servislerde tercih edilirken, yüksek hacimli veya esnek şema gerektiren servislerde NoSQL çözümleri kullanılabilmektedir (Han ve ark., 2011).

Bununla birlikte, bağımsız veritabanları yaklaşımı veri tutarlılığı ve servisler arası veri paylaşımı açısından ek tasarım zorlukları doğurmaktadır. Literatürde mikroservis mimarilerinde güçlü tutarlılık yerine çoğunlukla eventual consistency yaklaşımının benimsendiği ifade edilmektedir (Kleppmann ve ark., 2018).

2.6. Dağıtık Arama Sistemleri ve Log Yönetimi

Mikroservis mimarilerinde servislerin bağımsız ve dağıtık şekilde çalışması, sistem genelinde izleme ve loglama ihtiyacını artırmaktadır. Artan servis sayısı,

geleneksel merkezi izleme yaklaşımlarının yetersiz kalmasına neden olmakta ve daha gelişmiş gözlemlenebilirlik çözümlerini zorunlu hâle getirmektedir.

Literatürde, mikroservis tabanlı sistemlerde merkezi loglama ve izleme mekanizmalarının sistem davranışlarının analiz edilmesi, hata tespit süreçlerinin hızlandırılması ve operasyonel farkındalığın artırılması açısından kritik bir rol oynadığı belirtilmektedir. Ayrıca, dağıtık izleme ve tracing yaklaşımlarının, servisler arası etkileşimlerin daha iyi anlaşılmasını sağladığı ve sistem güvenilirliğini artırdığı ifade edilmektedir. Bu tür dağıtık arama ve analiz sistemleri, özellikle yüksek trafikli ve karmaşık mikroservis mimarilerinde sistem gözlemlenebilirliğini artıran önemli bir bileşen olarak değerlendirilmektedir.

2.7. Micro-Frontend Mimarisi

Micro-frontend mimarisi, mikroservis yaklaşımının kullanıcı arayüzü katmanına uygulanmış hâli olarak tanımlanmaktadır. Bu mimaride, geleneksel olarak tek parça hâlinde geliştirilen ön yüz uygulamaları; bağımsız geliştirilebilir ve dağıtılabılır alt uygulamalara ayrılmaktadır. Her micro-frontend genellikle belirli bir iş alanını temsil etmekte ve kendi geliştirme sürecine sahip olmaktadır (Camara ve ark., 2021).

Literatürde micro-frontend mimarisinin temel amacı, büyük ölçekli web uygulamalarında ön yüz kod tabanının zamanla karmaşık hâle gelmesini engellemek ve ekipler arası bağımlılığı azaltmaktır. Bu yaklaşım, ekiplerin paralel çalışmasını kolaylaştırmakta ve geliştirme süreçlerinin daha yönetilebilir olmasına katkı sağlamaktadır.

Akademik çalışmalarda micro-frontend mimarisinin, ön yüz bileşenlerinin bağımsız dağıtılabilmesi sayesinde sistem genelinde büyük sürüm geçişlerine olan ihtiyacı azalttığı belirtilmektedir. Bununla birlikte, istemci tarafında artan bileşen sayısının performans sorunlarına yol açabileceği ve ortak bileşen yönetiminin dikkatli tasarlanması gerektiği vurgulanmaktadır (Camara ve ark., 2021).

2.8. Konteynerizasyon ve Orkestrasyon (Kubernetes)

Yazılım servislerinin bağımsız birimler halinde paketlenmesi ve yönetilmesi, modern mikroservis mimarilerinin temel taşıdır. Literatürde Docker gibi konteyner teknolojilerinin, uygulamaların farklı ortamlarda tutarlı çalışmasını sağladığı ve kaynak

kullanımını optimize ettiği vurgulanmaktadır. Bu konteynerlerin binlerce örneğinin yönetilmesi, ölçeklendirilmesi ve hata durumunda yeniden başlatılması ise Kubernetes gibi orkestrasyon araçları ile sağlanmaktadır. Akademik çalışmalarda, Kubernetes'in "self-healing" ve "auto-scaling" özelliklerinin sistem sürdürülebilirliği üzerindeki pozitif etkileri sıklıkla ele alınmaktadır (Bernstein, 2014).

2.9. Hizmet Ağı Mimarisi

Mikroservis sayısı arttıkça, servisler arası güvenli iletişim, trafik yönetimi ve gözlemlenebilirlik karmaşık bir hal almaktadır. Literatürde bu karmaşıklığı çözmek için "Service Mesh" katmanı önerilmektedir. Istio veya Linkerd gibi araçlarla uygulanan bu mimari, iletişim mantığını iş mantığından ayırarak mTLS tabanlı güvenlik ve gelişmiş trafik yönlendirme imkanları sunar. Yapılan çalışmalarda, Service Mesh yapısının uygulama koduna dokunmadan ağ seviyesinde denetim sağladığı ifade edilmektedir (Li ve ark., 2019).

2.10. Sürekli Entegrasyon ve Sürekli Dağıtım (CI/CD)

Çevik yazılım geliştirme süreçlerinde, kodun otomatik olarak test edilmesi ve yayına alınması kritik öneme sahiptir. CI/CD boru hatları (pipelines), insan hatasını minimize ederek yazılım kalitesini artırmaktadır. Literatürde, mikroservislerin bağımsız dağıtılabilir doğasının ancak güçlü bir CI/CD altyapısı ile verimli kullanılabileceği belirtilmektedir. Otomatik testlerin bu süreçteki rolü, teknik borcun birikmesini engelleyen en önemli mekanizma olarak değerlendirilmektedir (Humble ve Farley, 2010).

2.11. Kimlik Doğrulama ve Yetkilendirme (OAuth2 ve JWT)

Dağıtık sistemlerde kullanıcı kimliğinin servisler arasında güvenli şekilde taşınması için JSON Web Token (JWT) ve OAuth2 protokolleri standart haline gelmiştir. Literatürde, "stateless" (durumsuz) yapısı nedeniyle JWT'nin mikroservislerde oturum yönetimini kolaylaştırdığı vurgulanmaktadır. API Gateway ile entegre çalışan merkezi kimlik doğrulama servislerinin, sistemin genel güvenlik yüzeyini daralttığı ve merkezi olmayan bir yetkilendirme kontrolü sağladığı ifade edilmektedir (Bertocci, 2019).

2.12. Altyapı Olarak Kod (Infrastructure as Code - IaC)

Bulut tabanlı sistemlerin ve mikroservis altyapılarının manuel olarak yapılandırılması, hata payını artırmakta ve tekrarlanabilirliği zorlaştırmaktadır. IaC yaklaşımı, sunucu, ağ ve veritabanı gibi altyapı bileşenlerinin tanımlayıcı dosyalar (Terraform, Ansible vb.) aracılığıyla yönetilmesini sağlar. Akademik çalışmalarda, IaC kullanımının felaket kurtarma (disaster recovery) süreçlerini hızlandırdığı ve altyapı sürümlemesini (versioning) mümkün kıldığı belirtilmektedir (Morris, 2016).

2.13. Yazılım Test Piramidi ve Test Otomasyonu

Mikroservis mimarilerinde sistem bütünlüğünü korumak için test stratejileri hayati önem taşır. Literatürde "Test Piramidi" modeli; çok sayıda birim test (unit test), orta düzeyde entegrasyon testi ve az sayıda uçtan uca (E2E) testin kombinasyonunu önermektedir. Özellikle servisler arası sözleşmelerin bozulmadığını teyit eden "Contract Testing" yaklaşımının, mikroservis ekosistemlerinde bağımsız dağıtımı güvence altına alan temel bir yöntem olduğu vurgulanmaktadır (Cohn, 2009).

2.14. Sunucusuz (Serverless) Hesaplama Modeli

Mikroservis evriminin bir sonraki adımı olarak görülen Serverless mimari, geliştiricilerin sunucu yönetimiyle ilgilenmeden sadece fonksiyonel kod (FaaS) yazmasına odaklanır. Literatürde, bu modelin "pay-as-you-go" (kullandığın kadar öde) maliyet avantajı ve sonsuz ölçeklenebilirlik sunduğu ifade edilmektedir. Bununla birlikte, "cold start" gecikmeleri ve servis sağlayıcıya bağımlılık (vendor lock-in) gibi dezavantajların dikkatli yönetilmesi gerektiği belirtilmektedir (Castro ve ark., 2019).

2.15. Gözlemlenebilirlik (Observability) ve Dağıtık İzleme

Dağıtık sistemlerde sadece log tutmak, bir hatanın kök nedenini bulmak için yeterli değildir. Gözlemlenebilirlik; metrikler, loglar ve izler (traces) olmak üzere üç temel sütuna dayanır. Literatürde, bir isteğin onlarca servis arasındaki yolculuğunu takip eden "Distributed Tracing" (Jaeger, Zipkin vb.) yöntemlerinin, performans darboğazlarını tespit etmedeki başarısı vurgulanmaktadır. Bu sistemler, mikroservislerin karmaşık etkileşim ağını görünür kılarak operasyonel farkındalık sağlar (Wambler ve ark., 2018).

3. MATERYAL VE YÖNTEM

Bu bölümde, e-ticaret platformunun tasarımında ve geliştirilmesinde kullanılan donanım ve yazılım materyalleri ile projenin hayata geçirilmesinde izlenen yöntemler detaylandırılmıştır. Çalışma kapsamında, modern yazılım mühendisliği prensipleri olan Mikroservis Mimarisi, Konteynerizasyon (Docker) ve Katmanlı Mimari yaklaşımları benimsenmiştir.

3.1. Materyaller

Mikroservis mimarisinin geliştirme ve test süreçleri, monolitik uygulamalara kıyasla sistem kaynakları üzerinde çok daha fazla yük oluşturmaktadır. Auth Service, User Service, Product Service gibi servislerin ve veritabanlarının Docker konteynerleri içerisinde eş zamanlı çalıştırılması gerektiğinden, geliştirme ortamının donanım kapasitesi kritik önem taşımaktadır.

Proje geliştirme sürecinde kullanılan iş istasyonunun teknik özellikleri Çizelge 3.1’de verilmiştir.

Çizelge 3.1: Sistem Geliştirme ve Test Donanımı Bileşenleri

Bileşen Grubu	Tanım/Özellik
İşlemci (CPU)	Intel® Core™ i7-12650H
Bellek (RAM)	16 GB DDR5 5600MHz
Depolama (SSD)	512 GB NVMe M.2 SSD
Ekran Kartı (GPU)	NVIDIA Geforce RTX 4060
Sanallaştırma Katmanı	WSL 2 Backend
Kod Editörü	Visual Studio 2026 / Fleet

3.1.1. Kullanılan teknoloji ve kütüphaneler

Geliştirilen e-ticaret platformunun backend mimarisi, yüksek performans, güvenlik ve gözlemlenebilirlik (observability) gereksinimlerini karşılamak üzere titizlikle seçilmiş NuGet paketleri üzerine inşa edilmiştir.

Sistemin veri güvenliği katmanında, kullanıcı parolalarının güvenli bir şekilde saklanması için BCrypt.Net-Next kütüphanesi ile "salting" ve "hashing" işlemleri uygulanmaktadır. Kimlik doğrulama süreçlerinde ise endüstri standardı olan JWT (JSON Web Token) yapısı, Microsoft.AspNetCore.Authentication.JwtBearer paketi ile yönetilmektedir. Ayrıca, veritabanı şifreleri ve API anahtarları gibi hassas verilerin kod

içerisinde açıkça tutulmasını engellemek amacıyla VaultSharp kütüphanesi kullanılarak HashiCorp Vault entegrasyonu sağlanmıştır.

Dağıtık mikroservis mimarisinin en kritik ihtiyaçlarından biri olan sistem izlenebilirliği için OpenTelemetry ve Prometheus kütüphaneleri projeye dahil edilmiştir. Bu sayede servisler arası isteklerin takibi (tracing) ve sistem metriklerinin (CPU, RAM, İstek Sayısı) toplanması otomatik hale getirilmiştir.

Veritabanı iletişimde Npgsql sürücüsü ile PostgreSQL entegrasyonu sağlanmış, loglama altyapısı için ise Serilog kütüphanesi tercih edilmiştir.

Proje genelinde kullanılan temel kütüphaneler, versiyonları ve kullanım amaçları Çizelge 3.2’te detaylandırılmıştır.

Çizelge 3.2: Sistem Bileşenleri ve Kullanılan Üçüncü Taraf Kütüphaneler

Kütüphane / Paket Adı	Versiyon	Kullanım Amacı
BCrypt.Net-Next	4.0.3	Kullanıcı parolalarının güvenli şekilde hashlenmesi.
JwtBearer	8.0.0	JWT tabanlı kimlik doğrulama mekanizması.
Npgsql.EFCore.PostgreSQL	8.0.0	PostgreSQL veritabanı iletişimi ve ORM yönetimi.
OpenTelemetry (Hosting/Exporter)	1.14.0	Dağıtık izleme (tracing) ve sistem gözlemlenebilirliği.
Serilog (AspNetCore/Seq)	10.0.0	Merkezi loglama ve Seq entegrasyonu.
VaultSharp	1.13.0.1	HashiCorp Vault üzerinden hassas veri yönetimi.
Prometheus-net.AspNetCore	8.2.2	Sistem metriklerinin toplanması ve izlenmesi.
HealthChecks.UI.Client	9.0.0	Servis sağlık durumlarının izlenmesi.

3.1.2. Yazılım ve geliştirme araçları

Projenin tasarım, kodlama ve test aşamalarında kullanılan tüm yazılım araçları; modern yazılım geliştirme standartlarına uygun, sürdürülebilir ve açık kaynak topluluğu tarafından desteklenen teknolojiler arasından seçilmiştir. Geliştirme sürecinin verimliliğini artırmak ve mikroservis mimarisinin gerektirdiği karmaşık yapıları yönetebilmek adına aşağıdaki temel geliştirme ortamları (IDE) ve yardımcı araçlar kullanılmıştır:

- **Visual Studio 2022:** Projenin backend tarafını oluşturan .NET tabanlı mikroservislerin (Auth, User, Product) kodlanması, derlenmesi ve yerel ortamda hata ayıklama (debugging) işlemleri için ana geliştirme ortamı olarak tercih edilmiştir.

- **Docker Desktop:** Mikroservislerin ve veritabanlarının yerel makinede sanallaştırılmış konteynerler olarak çalıştırılmasını sağlayan motor görevini üstlenmiştir.
- **Postman:** Geliştirilen RESTful API uç noktalarına (endpoints) HTTP istekleri (GET, POST, PUT, DELETE) gönderilerek servislerin doğruluğu test edilmiştir.
- **Swagger (OpenAPI):** Geliştirilen API'lerin standartlara uygun dokümantasyonunun otomatik oluşturulması ve tarayıcı üzerinden interaktif olarak test edilebilmesi için entegre edilmiştir.
- **Seq:** Dağıtık yapıdaki mikroservislerden gelen log kayıtlarının (hata, bilgi, uyarı) merkezi bir sunucuda toplanması ve analiz edilmesi amacıyla kullanılmıştır.

Proje altyapısında kullanılan kütüphaneler, çerçeveler (frameworks) ve veritabanı teknolojileri, sürümleri ve kullanım amaçlarıyla birlikte Çizelge 3.3'de detaylandırılmıştır.

Çizelge 3.3: Proje Geliştirme Sürecinde Kullanılan Yazılım Araçları ve Teknolojiler

Yazılım / Araç	Kullanım Amacı
Visual Studio 2022	.NET mikroservislerin kodlanması, derlenmesi ve hata ayıklama işlemleri.
Docker Desktop	Konteynerizasyon ve servislerin izole ortamlarda çalıştırılması.
Postman	API uç noktalarının manuel testi ve HTTP istek yönetimi.
Swagger (OpenAPI)	İnteraktif API dokümantasyonu ve tarayıcı tabanlı test arayüzü.
Seq	Merkezi log toplama, filtreleme ve gerçek zamanlı hata analizi.
DBeaver / Azure Data Studio	PostgreSQL veritabanı yönetimi ve SQL sorgularının yürütülmesi.

3.1.3. Veritabanı ve veri yönetim sistemleri

Geleneksel monolitik mimarilerde genellikle tek bir ilişkisel veritabanı (RDBMS) tüm yükü üstlenirken, bu çalışmada benimsenen mikroservis mimarisi gereği "Polyglot Persistence" (Çok biçimli veri saklama) yaklaşımı uygulanmıştır. Bu yaklaşım, her mikroservisin işlediği verinin yapısına (ilişkisel, doküman tabanlı, anahtar-değer veya vektörel) en uygun veritabanı teknolojisinin seçilmesine olanak tanımıştır.

Proje kapsamında dört farklı veritabanı yönetim sistemi (VTYS) hibrit bir yapıda kullanılmıştır:

PostgreSQL: Veri tutarlılığının (ACID prensipleri) kritik olduğu Kullanıcı (User) ve Ürün (Product) servislerinde, ilişkisel verilerin güvenli saklanması amacıyla ana veritabanı olarak kullanılmıştır.

MongoDB: Şema bağımsız (schemaless) yapısı sayesinde, ürünlere ait esnek özelliklerin (Attributes) ve sistem loglarının JSON formatında, yüksek okuma/yazma hızıyla depolanması için tercih edilen NoSQL veritabanıdır.

Redis: Sık erişilen verilerin (Örn: Ürün kategorileri, kullanıcı oturum tokenları) disk yerine RAM üzerinde tutularak milisaniyeler içinde sunulması (Caching) amacıyla kullanılmıştır.

Qdrant: Yapay zeka servisinin (AI Service) kalbini oluşturan bu vektör veritabanı, ürünlerin ve kullanıcı davranışlarının matematiksel vektörlere (Embeddings) dönüştürülerek saklanmasını sağlar. Bu sayede "benzer ürün bulma" veya "öneri sistemi" gibi işlemler klasik SQL sorgularına göre çok daha hızlı ve akıllıca gerçekleştirilmiştir.

Sistemde kullanılan veritabanı teknolojileri ve görev dağılımları Çizelge 3.4'te detaylandırılmıştır.

Çizelge 3.4: Servis Bazlı Veritabanı Seçimleri ve Kullanım Amaçları

Veritabanı Sistemi	Türü / Yapısı	Görevi ve Servis Entegrasyonu
PostgreSQL	Tek bir büyük bütün (Single Unit)	Bağımsız servisler (Auth, User, vb.)
MongoDB	Tüm uygulamanın yeniden yayını	Sadece ilgili servisin güncellenmesi
Redis	Tüm sistemin kopyalanması	İhtiyaç duyan servisin (örn: AI) ölçeklenmesi
Qdrant	Bir hata tüm sistemi durdurabilir	Bir servisteki hata diğerlerini etkilemez

3.2. Yöntem ve Mimari Tasarım

Bu çalışmada, e-ticaret platformunun tasarımında geleneksel monolitik yaklaşım yerine, modern yazılım mühendisliğinin güncel paradigması olan Mikroservis Mimarisi benimsenmiştir. Sistemin tasarım sürecinde, sürdürülebilirlik, ölçeklenebilirlik ve hata izolasyonu (fault isolation) temel hedefler olarak belirlenmiştir.

3.2.1. Mikroservis mimarisi yaklaşımı

Günümüz e-ticaret sistemlerinin gerektirdiği yüksek erişilebilirlik, anlık trafik artışlarına yanıt verebilme ve sürekli entegrasyon (CI/CD) ihtiyaçları, geleneksel yazılım mimarilerinin sınırlarını zorlamaktadır. Bu bağlamda, projenin temel tasarım kararı olarak, uygulamayı tek bir bütün (monolitik) olarak geliştirmek yerine, Mikroservis Mimarisi benimsenmiştir.

Geleneksel monolitik mimaride; kullanıcı arayüzü, iş mantığı ve veri erişim katmanları tek bir çalıştırılabilir birim (deployment unit) içerisinde paketlenir. Bu durum, uygulamanın herhangi bir modülünde yapılan küçük bir değişikliğin, tüm sistemin yeniden derlenmesini ve dağıtılmasını gerektirmesine yol açmaktadır. Ayrıca, sistemin sadece belirli bir modülüne (örneğin ödeme sistemi) yük bindiğinde dahi, tüm uygulamanın kopyasının oluşturulması (scaling) zorunluluğu, kaynak verimliliğini düşürmektedir.

Bu çalışmada uygulanan mikroservis yaklaşımında ise sistem, "tek bir işi yapan ve o işi iyi yapan" bağımsız servislere (Auth, User, Product, AI) bölünmüştür. Çizelge 3.5'te monolitik ve mikroservis mimarisi karşılaştırılmıştır.

Çizelge 3.5: Monolitik ve Mikroservis Mimarisi Karşılaştırılması

Karşılaştırma Ölçütü	Monolitik Mimari	Mikroservis Mimarisi
Yapısal Birim	Tek bir büyük bütün (Single Unit)	Bağımsız servisler (Auth, User, vb.)
Dağıtım (Deployment)	Tüm uygulamanın yeniden yayını	Sadece ilgili servisin güncellenmesi
Ölçeklenebilirlik	Tüm sistemin kopyalanması	İhtiyaç duyan servisin (örn: AI) ölçeklenmesi
Hata İzolasyonu	Bir hata tüm sistemi durdurabilir	Bir servisteki hata diğerlerini etkilemez
Teknoloji Yığılı	Tek bir dil/framework zorunluluğu	Her servis için en uygun teknoloji (Polyglot)
Veritabanı	Merkezi, tek veritabanı	Servise özel bağımsız veritabanları

Projede mikroservis mimarisinin tercih edilmesinin temel nedenleri ve sağladığı avantajlar şunlardır:

Bağımsız Ölçeklenebilirlik (Scalability): E-ticaret platformunda kampanya dönemlerinde sadece "Ürün Görüntüleme" ve "Sepet" servislerine yük binmektedir. Mikroservis mimarisi sayesinde, tüm sistemi büyütmek yerine sadece product-service ve basket-service konteynerlerinin sayısı artırılarak (horizontal scaling) kaynak optimizasyonu sağlanmıştır.

Teknoloji Çeşitliliği (Polyglot Architecture): Monolitik yapıda tüm sistemin aynı dille yazılması zorunluluğu varken, bu projede her servis problemin doğasına en uygun teknoloji ile geliştirilmiştir. Örneğin; yüksek işlem hacmi gerektiren backend servisleri .NET 8 ile yazılırken, veri analitiği ve makine öğrenmesi yetenekleri gerektiren öneri motoru (AI Service) için Python tercih edilmiştir.

Hata İzolasyonu (Fault Isolation): Monolitik bir yapıda oluşabilecek bellek sızıntısı (memory leak) tüm sunucuyu kilitleyebilirken; mikroservis mimarisinde ai-service üzerinde oluşabilecek bir hata, kullanıcıların sisteme giriş yapmasını (auth-service) veya ürünleri listelemesini engellememektedir. Bu durum sistemin genel erişilebilirliğini (availability) artırmaktadır.

Çevik Geliştirme ve Dağıtım: Her servis kendi kod tabanına (codebase) ve yaşam döngüsüne sahiptir. Bu sayede, "Kullanıcı Servisi"nde yapılan bir güncelleme, diğer servisleri durdurmadan veya etkilemeden canlı ortama alınabilmektedir.

3.2.2. Etki alanı odaklı tasarım

Mikroservis mimarisinin başarısı, servis sınırlarının ne kadar doğru belirlendiğine bağlıdır. Servislerin yanlış ayrıştırılması, ilerleyen süreçlerde servisler arası aşırı bağımlılığa ve yönetilemez bir yapıya neden olabilir. Bu riski minimize etmek amacıyla, projenin tasarım aşamasında Etki Alanı Odaklı Tasarım (Domain Driven Design - DDD) metodolojisi izlenmiştir.

DDD yaklaşımının temel taşı olan "Sınırlandırılmış Bağlam" (Bounded Context) prensibi gereği, e-ticaret iş modeli tek bir karmaşık bütün olarak değil, kendi içinde tutarlı ve özerk alt alanlara (sub-domains) ayrılmıştır. Her bir bağlam, kendi veritabanı şemasına, iş kurallarına ve ubiquitous (ortak) diline sahiptir.

Bu çalışma kapsamında belirlenen temel etki alanları ve sorumlulukları şunlardır:

Kimlik Yönetimi Bağlamı (Auth Context): Sistemin güvenlik kapısıdır. Kullanıcıların doğrulanması (Authentication), erişim yetkilerinin belirlenmesi (Authorization) ve güvenli oturum anahtarlarının (JWT) üretilmesi bu bağlamın sorumluluğundadır. Diğer hiçbir servis kullanıcı şifrelerini bilmez veya doğrulamaz; sadece bu bağlamdan gelen onay bilecini (Token) kabul eder.

Katalog ve Stok Bağlamı (Product Context): E-ticaretin temel varlığı olan "Ürün" nesnesinin yaşam döngüsünü yönetir. Ürünlerin eklenmesi, kategorize edilmesi, fiyatlandırma stratejileri ve stok takibi bu alanın sınırları içerisindedir.

Müşteri Bağlamı (User Context): Kullanıcının kişisel profili, adres defteri ve iletişim tercihleri gibi "Müşteri" varlığına ait verileri yönetir. Kimlik doğrulama bağlamından farklı olarak, burada kullanıcının sisteme girişi değil, sistemdeki varlığı ve özellikleri esas alınır.

Zeka ve Öneri Bağlamı (Intelligence Context): Sistemin analitik beynini oluşturur. Kullanıcıların geçmiş hareketleri ve ürün özellikleri arasındaki gizli ilişkileri analiz ederek, kişiye özel ürün önerileri sunar. Bu bağlam, operasyonel işlerden ziyade veri madenciliği ve çıkarım süreçlerine odaklanır.

Bu ayrıştırma stratejisi sayesinde, örneğin "Ürün" kavramının "Katalog" bağlamındaki anlamı ile "Sepet" bağlamındaki anlamı birbirinden izole edilmiştir. Böylece bir bağlamda yapılan model değişikliğinin, diğer bağlamları zorunlu değişikliğe sürüklemesi engellenmiştir.

3.2.3. Servis içi mantıksal katmanlama

Mikroservislerin her biri, kod karmaşıklığını önlemek ve bakım kolaylığı sağlamak amacıyla node.js mimarisinin standartlarına uygun olarak mantıksal katmanlara ayrılmıştır. Bu yapıda, veri tabanı nesneleri, iş kuralları ve dış dünyaya açılan API uç noktaları birbirinden ayrıştırılmıştır.

Projedeki servislerin iç mimarisi temel olarak üç ana bileşenden oluşmaktadır:

Veri Modelleri (Entities / Models): Servisin yönettiği verilerin şemasını belirleyen sınıflardır. Örneğin Ürün Servisi içerisinde yer alan Product.cs sınıfı, veritabanındaki Products tablosunun bir yansımasıdır. Bu katman, projenin en saf halidir ve veritabanı yapısını temsil eder.

Veri Erişim Katmanı (Data Access / Context): Uygulamanın veritabanı ile konuştuğu bölümdür. Entity Framework Core (EF Core) kütüphanesinden türetilen DbContext sınıfları burada yer alır. SQL sorguları yazmak yerine, C# kodları (LINQ) kullanılarak veritabanına veri ekleme, silme ve güncelleme işlemleri bu katman üzerinden yönetilir. Ayrıca veritabanı değişikliklerinin (Migrations) yönetimi de burada yapılır.

API Denetleyicileri (Controllers): Dış dünyadan veya diğer servislerden gelen HTTP isteklerini (GET, POST, PUT, DELETE) karşılayan en dış katmandır.

İstemciden gelen veriyi alır.

Gerekli doğrulamaları yapar.

Veritabanı katmanını çağırarak işlemi gerçekleştirir.

Sonucu JSON formatında (200 OK, 404 Not Found vb.) geri döndürür.

Bu yapı sayesinde, bir servis içerisindeki veritabanı kodları ile API yönlendirme kodları birbirine karışmamış, sade ve yönetilebilir bir kod tabanı (codebase) elde edilmiştir.

3.3. Veri Yönetimi ve İletişim

Mikroservis mimarisinin en kritik tasarım kararlarından biri, verinin nasıl saklanacağı ve servisler arasında nasıl taşınacağıdır. Bu çalışmada, verilerin tutarlılığını ve güvenliğini sağlamak amacıyla servisler arası veri izolasyonu ilkesi benimsenmiş ve iletişim standartları buna göre belirlenmiştir.

3.3.1. Servis başına veritabanı yaklaşımı

Geleneksel monolitik mimarilerde, uygulamanın tüm modülleri (Kullanıcı, Ürün, Sipariş vb.) genellikle tek bir merkezi veritabanını (Shared Database) ortaklaşa kullanır. Bu durum, veri tutarlılığını sağlamayı kolaylaştırır da, sistem büyüdükçe veritabanında darboğazlara (bottleneck) ve servisler arasında sıkı bir bağımlılığa (tight coupling) neden olmaktadır.

Bu çalışmada, mikroservislerin özerkliğini korumak ve sistemin ölçeklenebilirliğini artırmak amacıyla "Database per Service" (Servis Başına Veritabanı) deseni uygulanmıştır. Bu desen gereği, her mikroservis sadece kendi sahipliğindeki veritabanına erişebilir; başka bir servisin veritabanındaki tablolara doğrudan SQL sorgusu atamaz.

Çizelge 3.6: Paylaşımlı veritabanı ve servis başına veritabanı yaklaşımlarının karşılaştırılması

Özellik	Paylaşımlı Veritabanı (Monolitik)	Servis Başına Veritabanı (Mikroservis)
Erişim	Tüm servisler her tabloya erişebilir.	Sadece ilgili servis kendi verisine erişir.
Bağımlılık	Sıkı bağımlılık (Tight Coupling).	Tam özerklik (Loose Coupling).
Hata Yayılımı	Veritabanı çökerse tüm sistem durur.	Bir veritabanı hatası sadece o servisi etkiler.
Ölçeklenebilirlik	Zor ve maliyetli.	Servis bazlı (yatayda) kolay ölçekleme.

Projede bu yaklaşımın tercih edilmesinin temel nedenleri şunlardır:

Teknolojik Özgürlük (Polyglot Persistence): Her veri tipi için en uygun veritabanı teknolojisi seçilebilmiştir. Örneğin; ilişkisel bütünlük gerektiren sipariş ve müşteri verileri için PostgreSQL kullanılırken, milisaniyeler seviyesinde erişim gerektiren sepet ve oturum bilgileri için Redis, yapay zeka vektör verileri için ise Qdrant kullanılmıştır.

Hata İzolasyonu: Tek bir merkezi veritabanı kullanıldığında, veritabanı sunucusunda oluşacak bir kilitlenme (deadlock) tüm sistemi durdurur. Servis başına

veritabanı yapısında ise, Ürün Servisi'nin veritabanında yaşanan bir kesinti, Kullanıcı Servisi'nin çalışmasını engellemez.

Bağımsız Veri Modelleme: Geliştirme sürecinde, "User Service" ekibi veritabanı şemasında (schema) değişiklik yaptığında, bu değişiklik diğer servislerin kodlarını kırmaz. Servisler arası iletişim sadece tanımlı API kontratları üzerinden sağlandığı için veritabanı yapısı dış dünyadan soyutlanmıştır.

3.3.2. Servisler arası iletişim ve güvenlik (JWT)

Mikroservis mimarilerinde servislerin birbirleriyle ve dış dünya (client) ile nasıl iletişim kuracağı, sistemin performansını ve güvenliğini doğrudan etkileyen en temel faktörlerden biridir. Bu tez çalışmasında geliştirilen e-ticaret platformunda, servisler arası iletişim protokolü olarak, endüstri standardı olan ve platform bağımsızlığı sağlayan HTTP/REST (Representational State Transfer) mimarisi benimsenmiştir. RESTful servislerin hafif yapısı ve web standartlarıyla tam uyumu, dağıtık sistemlerde entegrasyonu kolaylaştırmaktadır.

Ancak, dağıtık mimarinin getirdiği en büyük zorluklardan biri, kullanıcı oturumlarının ve yetkilendirme süreçlerinin güvenli bir şekilde yönetilmesidir. Monolitik uygulamalarda yaygın olarak kullanılan sunucu tarafı "Session" (Oturum) yönetimi, mikroservis mimarisinde uygulanamaz. Çünkü servisler "Stateless" (Durumsuz) çalışmak zorundadır; yani bir servis, gelen isteğin sahibini tanımak için kendi hafızasında bir oturum bilgisi tutmamalıdır. Bu durum, sistemin yatayda ölçeklenebilmesi (Horizontal Scaling) için bir zorunluluktur.

Bu gereksinimleri karşılamak amacıyla, projenin güvenlik ve kimlik doğrulama katmanında JSON Web Token (JWT) standardı (RFC 7519) kullanılmıştır. JWT, taraflar arasında güvenli bir şekilde JSON nesnesi olarak bilgi alışverişi yapılmasını sağlayan kompakt ve URL-safe bir yöntemdir.

JWT Tabanlı Kimlik Doğrulama Akışı

Sistemdeki güvenlik mimarisi, merkezi bir kimlik doğrulama otoritesi (Auth Service) üzerine kurgulanmıştır. Kullanıcıların sisteme giriş yapması ve korumalı kaynaklara (örneğin; sipariş geçmişini görüntüleme veya yeni sipariş oluşturma) erişmesi sürecinde "Token Based Authentication" (Token Tabanlı Kimlik Doğrulama) yöntemi izlenmektedir.

Sistemin kimlik doğrulama ve yetkilendirme akışı şu adımlardan oluşmaktadır:

Kimlik Doğrulama İsteği: İstemci (Web veya Mobil Arayüz), kullanıcı adı ve şifresini içeren bir POST isteğini Auth Service'e gönderir.

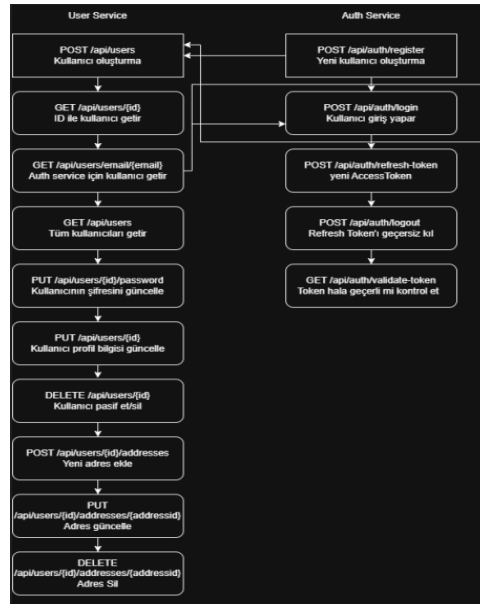
Doğrulama ve Token Üretimi: Auth Service, veritabanından kullanıcı bilgilerini doğrular. Bilgiler doğruysa, kullanıcının ID'sini, rollerini (Admin, User vb.) ve token'ın geçerlilik süresini (Expiration Time) içeren şifrelenmiş bir Access Token (Erişim Anahtarı) üretir. Bu token, sunucunun özel anahtarı (Secret Key) ile imzalanır.

Token'ın İletilmesi: Üretilen JWT, yanıt gövdesinde (Response Body) istemciye gönderilir. İstemci bu token'ı yerel depolama alanında (Local Storage veya Cookie) saklar.

Korumalı Kaynağa Erişim: İstemci, diğer mikroservislere (Örn: Product Service, Order Service) istek atarken, sahip olduğu bu token'ı HTTP başlığının (Header) Authorization alanına Bearer <token> formatında ekler.

Token Doğrulama: İsteği alan mikroservis, token'ın imzasını ve geçerlilik süresini kontrol eder. Token geçerliyse, içerisindeki "Claims" (Kullanıcı yetkileri) okunur ve işlem gerçekleştirilir.

Aşağıdaki şekilde, sistemde uygulanan bu token tabanlı iletişim ve güvenlik akışı şematize edilmiştir.



Şekil 3.6: Auth ve User Servisleri API Uç Noktaları ve İletişim Akışı

Güvenlik Katmanının Teknik Detayları

Projede kullanılan JWT yapısı üç temel bölümden oluşmaktadır: Header (Başlık), Payload (Veri) ve Signature (İmza).

Header: Kullanılan şifreleme algoritmasını (bu projede HMAC SHA256) ve token tipini belirtir.

Payload: Kullanıcıya ait "Claims" olarak adlandırılan bilgileri taşır. Projede Payload içerisinde güvenlik gereği kullanıcının şifresi gibi hassas veriler taşınmamakta; sadece UserId, Email ve Role bilgileri ile token'ın son kullanma tarihi (exp) yer almaktadır. Bu sayede, token ele geçirilse bile kritik verilere erişim engellenmiş olur.

Signature: Token'ın içeriğinin yolda değiştirilip değiştirilmediğini garanti altına alan kısımdır. Auth Service tarafında saklanan gizli bir anahtar (Secret Key) ile oluşturulur. Diğer servisler, gelen isteğin güvenilirliğini bu imzayı kontrol ederek doğrular.

Bu mimari sayesinde, her mikroservisin kendi içinde kullanıcı doğrulama (login) mekanizması kurmasına gerek kalmamıştır. Servisler sadece gelen token'ın geçerliliğini kontrol ederek, iş mantıklarına odaklanabilmektedir. Ayrıca, token içerisinde taşınan rol bilgileri sayesinde, örneğin sadece "Admin" rolüne sahip kullanıcıların ürün silebilmesi gibi yetkilendirme (Authorization) kontrolleri de servis seviyesinde kolaylıkla sağlanmıştır.

3.4. Konteynerizasyon (Docker)

Geleneksel yazılım dağıtım süreçlerinde sıkça karşılaşılan "Benim bilgisayarında çalışıyordu, sunucuda çalışmadı" sorunu, mikroservis mimarilerinde çok daha kritik bir hal almaktadır. Onlarca farklı servisin, farklı bağımlılıklarla (dependencies) ve farklı ortam değişkenleriyle tutarlı bir şekilde çalıştırılması gerekmektedir. Bu tez çalışmasında, bu sorunu kökten çözmek ve uygulama bağımlılıklarını izole etmek amacıyla Docker konteynerizasyon teknolojisi kullanılmıştır.

Sanal makinelerin (VM) aksine, donanım sanallaştırması yerine işletim sistemi seviyesinde sanallaştırma sağlayan Docker, her bir mikroservisin (Auth, Product, Basket vb.) kendi kütüphaneleri ve çalışma zamanı (runtime) ile paketlenmesini sağlar. Bu sayede, geliştirme ortamı (Development) ile canlı ortam (Production) arasındaki farklar ortadan kaldırılmış ve taşınabilirlik (Portability) maksimize edilmiştir.

3.4.1. Docker stratejisi ve Çok aşamalı derleme

Proje kapsamında her bir mikroservis için kök dizinde bir "Dockerfile" oluşturulmuştur. Bu dosyalar, ilgili servisin nasıl derleneceğini, hangi portu dinleyeceğini ve hangi ortam değişkenlerini kullanacağını tanımlayan bir reçete niteliğindedir.

Docker imajlarının boyutu ve güvenliği, mikroservislerin dağıtım hızı (Deployment Speed) için kritik öneme sahiptir. Bu projede, imaj boyutlarını minimize etmek ve gereksiz derleme araçlarını canlı ortama taşımamak adına "Multi-Stage Build" (Çok Aşamalı Derleme) stratejisi uygulanmıştır. Bu strateji iki temel aşamadan oluşur:

Derleme Aşaması (Build Stage): Bu aşamada, uygulamanın kaynak kodlarını derlemek için gerekli olan tüm SDK (Software Development Kit) araçlarını içeren geniş kapsamlı bir temel imaj (mcr.microsoft.com/dotnet/sdk:8.0) kullanılır. Kodlar bu geçici konteyner içinde derlenir (dotnet build) ve yayınlanabilir hale (dotnet publish) getirilir.

Çalışma Aşaması (Runtime Stage): İkinci aşamada ise, içinde sadece uygulamayı çalıştırmak için gerekli olan minimum dosyaları barındıran, çok daha hafif bir imaj (mcr.microsoft.com/dotnet/aspnet:8.0) temel alınır. İlk aşamada üretilen derlenmiş dosyalar (DLL'ler), bu hafif imaja kopyalanır.

Bu yöntem sayesinde, içinde kaynak kodların veya ağır derleme araçlarının bulunmadığı, sadece çalışan uygulamanın yer aldığı, güvenli ve optimize edilmiş (yaklaşık 100-200 MB boyutunda) Docker imajları elde edilmiştir. Bu durum, sunucu kaynaklarının (CPU/RAM) verimli kullanılmasını ve servislerin saniyeler içinde ayağa kalkmasını sağlamaktadır.

3.4.2. Konteyner orkestrasyonu (Docker Compose)

Mikroservis mimarisindeki en büyük zorluklardan biri, çok sayıdaki konteynerin birbiriyle uyumlu bir şekilde başlatılması ve ağ iletişiminin yönetilmesidir. Örneğin, ProductService ayağa kalkmadan önce RabbitMQ veya SQL Server konteynerlerinin hazır olması gerekebilir. Bu karmaşıklığı yönetmek için projenin kök dizinde bir "docker-compose.yml" dosyası yapılandırılmıştır.

Docker Compose, YAML formatındaki tek bir konfigürasyon dosyası üzerinden çoklu konteyner uygulamalarının tanımlanmasını ve çalıştırılmasını sağlar. Projede Docker Compose şu amaçlarla kullanılmıştır:

Servis Tanımları: Her bir mikroservis, veritabanı ve yardımcı araçlar (Homer Dashboard, RabbitMQ vb.) birer "service" olarak tanımlanmıştır.

Ağ Yönetimi (Networking): Konteynerlerin birbirleriyle IP adresi yerine servis isimleriyle (DNS Resolution) haberleşebilmesi için ortak bir "Bridge Network" oluşturulmuştur. Örneğin, Web UI servisi, Ürün Servisi'ne erişmek için `http://localhost:5000` yerine doğrudan `http://productservice:80` adresini kullanabilmektedir.

Bağımlılık Yönetimi (Depends On): Servislerin başlatılma sırası `depends_on` parametresi ile kontrol altına alınmıştır. Böylece veritabanı ayağa kalkmadan servisin çalışmaya başlayıp hata vermesi engellenmiştir.

Veri Kalıcılığı (Volumes): Veritabanı konteynerleri kapatılıp açıldığında verilerin kaybolmaması için "Docker Volumes" kullanılarak veriler fiziksel diske map edilmiştir (eşlenmiştir).

Aşağıdaki şemada, Docker Compose ile ayağa kaldırılan sistemin ağ topolojisi ve konteynerler arası iletişim yapısı gösterilmektedir:

[Şekil 3.X: Docker Compose Ağ Yapısı ve Konteyner İletişim Şeması] (Buraya; ortada bir Docker Network, etrafında Database, RabbitMQ ve senin C# servislerinin kutucuklar halinde birbirine bağlı olduğu bir şema koyacaksın)

Bu orkestrasyon yapısı sayesinde, geliştirici tek bir komutla (`docker-compose up -d`) tüm e-ticaret platformunu, veritabanlarını ve arayüzleri, herhangi bir kurulum karmaşası yaşamadan saniyeler içinde çalışır hale getirebilmektedir.

3.5. Merkezi İzleme ve Erişim Paneli

Mikroservis mimarisine sahip projelerde, geliştirme ve test süreçlerindeki en büyük pratik zorluklardan biri, çok sayıdaki servisin ve yardımcı aracın (Database, RabbitMQ, Redis vb.) farklı portlar üzerinde çalışmasıdır. Örneğin; ProductService 5001 portunda, RabbitMQ yönetim paneli 15672 portunda çalışırken, bu adreslerin ve portların tamamını akılda tutmak veya sürekli dökümantasyondan kontrol etmek, geliştirme sürecini yavaşlatmaktadır.

Bu çalışmada, tüm servislerin durumunu tek bir noktadan gözlemlemek ve ilgili servislere hızlı erişim sağlamak amacıyla, açık kaynak kodlu ve hafif bir statik dashboard (panosu) uygulaması olan Homer kullanılmıştır. Homer, karmaşık bir veritabanı kurulumuna ihtiyaç duymadan, sadece basit bir konfigürasyon dosyası ile yönetilebilen, Docker ile tam uyumlu ve modern bir arayüz sunan bir "başlangıç sayfası" (homepage) çözümüdür. Sistem yöneticisi veya geliştirici, tarayıcı üzerinden tek bir adrese (Örn:

http://localhost:8080) giderek, ayağa kaldırılmış tüm mikroservisleri ve altyapı araçlarını kategorize edilmiş halde görebilir ve tek tıkla erişebilir.

3.5.1. Docker stratejisi ve Çok aşamalı derleme

Homer'ın çalışma mantığı, YAML (YAML Ain't Markup Language) formatında yazılan tek bir konfigürasyon dosyasına (config.yml) dayanır. Bu dosya, dashboard üzerinde görünecek başlıkları, ikonları, servis gruplarını ve yönlendirme linklerini tanımlar. Homer konteyneri ayağa kalktığında bu dosyayı okur ve statik HTML sayfasını otomatik olarak oluşturur.

Proje kapsamında oluşturulan dashboard yapısı, sistemin mantıksal katmanlarına göre gruplandırılmıştır:

Altyapı (Infrastructure): RabbitMQ Management, SQL Server, Redis gibi temel bileşenler.

Mikroservisler (Microservices): Product, Order, Auth gibi backend servislerinin Swagger dokümantasyon adresleri.

Uygulamalar (Applications): Son kullanıcının etkileşime girdiği Web UI arayüzü.

Aşağıda, proje için yapılandırılan config.yml dosyasından, servislerin nasıl tanımlandığını gösteren örnek bir kesit yer almaktadır;

```
- name: "Redis"
  subtitle: "Sepet & Cache"
  url: "http://localhost:8084"
  logo: "https://cdn.worldvectorlogo.com/logos/redis.svg"

- name: "Qdrant"
  subtitle: "AI Vektör Hafızası"
  url: "http://localhost:6333/dashboard"
  logo: "https://qdrant.tech/img/qdrant-logo.svg"

- name: "Altyapı & İzleme"
  icon: "fas fa-tools"
  items:
    - name: "Status (Kuma)"
      subtitle: "Canlı İzleme"
      url: "http://localhost:5011"
      logo: "https://raw.githubusercontent.com/louislam/uptime-kuma/master/public/icon.svg"
```

Şekil 3.7: Config.yml dosyası

4. ARAŞTIRMA SONUÇLARI VE TARTIŞMA

Bu bölümde, tez çalışması kapsamında geliştirilen ve Fuiira E-Commerce olarak adlandırılan çok satıcılı e-ticaret platformunun uygulama katmanına dair teknik detaylar ele alınmaktadır. Mevcut çalışma, teorik düzlemde kurulan sistem mimarisinin pratik uygulama süreçlerini, veri modelleme stratejilerini ve sistem bileşenleri arasındaki haberleşme protokollerini teknik bir perspektifle sunar. Yazılımın arka uç ve ön yüz birimlerinin entegrasyon süreçleri ile iş mantığının kaynak kod düzeyindeki karşılıkları, sistemin çalışma prensiplerini açıklamak amacıyla detaylandırılmıştır.

Sistemin ana omurgasını, Node.js ortamında Express.js çerçevesi kullanılarak geliştirilen ve Prisma ORM katmanı ile veritabanı yönetimini sağlayan storebackend uygulaması oluşturmaktadır. Bu merkezi yapı, platformun veri tutarlılığını ve işlem yönetimini koordine etmektedir. Kullanıcı deneyimini yönetmek adına sistem; müşteri mağazası, platform yönetim merkezi ve satıcı paneli olmak üzere üç temel kullanıcı arayüzü ile desteklenmektedir. Bu arayüzlerin geliştirme sürecinde React ekosistemi tercih edilmiş, modern yapılandırma ve performans optimizasyonu gereksinimleri doğrultusunda Vite paketleme aracı kullanılmıştır.

Proje mimarisi içerisinde kimlik doğrulama, ürün yönetimi ve sipariş süreçlerini kapsayan auth-service, product-service ve order-service gibi özelleşmiş servis klasörleri bulunmaktadır. Bu yapılandırma, sistemin gelecekte mikroservis mimarisine evrilmesine olanak tanıyacak bir modülerlik sunmaktadır. Ancak, bu tez çalışması kapsamında gerçekleştirilen işlevsel analizler ve entegrasyon süreçlerinin anlatımı, tüm sistemin operasyonel merkezi konumunda olan storebackend uygulaması temel alınarak yürütülmektedir. Uygulanan bu mimari yaklaşım, veri akışının tek bir merkezden denetlenmesini sağlarken, arayüzler arası veri senkronizasyonunun en düşük gecikme süresiyle gerçekleştirilmesini hedeflemektedir.

4.1. Kapsam ve yaklaşım

Platform bünyesinde tanımlanan kullanıcı rolleri ve bu rollerin sistem üzerindeki yetki sınırları, uçtan uca bir e-ticaret iş akışını yönetecek şekilde hiyerarşik olarak yapılandırılmıştır. Son kullanıcı tarafında sistem; ürünlerin dinamik listelenmesi, gelişmiş filtreleme seçenekleriyle sepete ekleme işlemleri, lojistik verilerin girişini takiben siparişin sonuçlandırılması ve kullanıcı profil paneli üzerinden geçmiş işlem

kayıtlarının izlenmesi süreçlerini kapsar. Satıcı rolü ise envanter yönetimi, stok takibi ve kendisine yönlendirilen sipariş bileşenlerinin gerçek zamanlı izlenmesini sağlayan özelleşmiş bir panel üzerinden operasyonlarını yürütür. Sistemin en üst katmanında yer alan yönetici rolü; kategori ağaçlarının oluşturulmasından ürün özellik sözlüklerinin tanımlanmasına, satıcı ve kullanıcı veritabanının denetlenmesinden platform geneli analitik istatistiklerin raporlanmasına kadar tüm ekosistemi merkezi bir otorite ile kontrol altında tutar.

Sistemde tanımlanan bu fonksiyonel süreçler, REST mimarisine uygun olarak kurgulanmış HTTP uç noktaları (endpoints) aracılığıyla PostgreSQL ilişkisel veritabanı üzerinde veri okuma ve yazma operasyonlarına dönüştürülür. Sistemin ölçeklenebilirliğini artırmak ve veritabanı üzerindeki yükü minimize etmek amacıyla, statik içerikler ve yüksek hacimli ürün görselleri veritabanı katmanında saklanmak yerine nesne depolama (object storage) çözümü olan MinIO üzerinde yapılandırılmıştır. Güvenlik mimarisi ise çok katmanlı bir savunma stratejisi üzerine inşa edilmiştir. Bu kapsamda, kullanıcı oturumlarının güvenli takibi için JSON Web Token (JWT) tabanlı kimlik doğrulama protokolü uygulanmış; sistem güvenliğini pekiştirmek amacıyla CORS (Cross-Origin Resource Sharing) politikaları, aşırı istek trafiğini önlemek için Rate Limiting mekanizmaları ve SQL enjeksiyonu gibi saldırı vektörlerine karşı girdi doğrulama (input sanitization) teknikleri devreye alınmıştır.

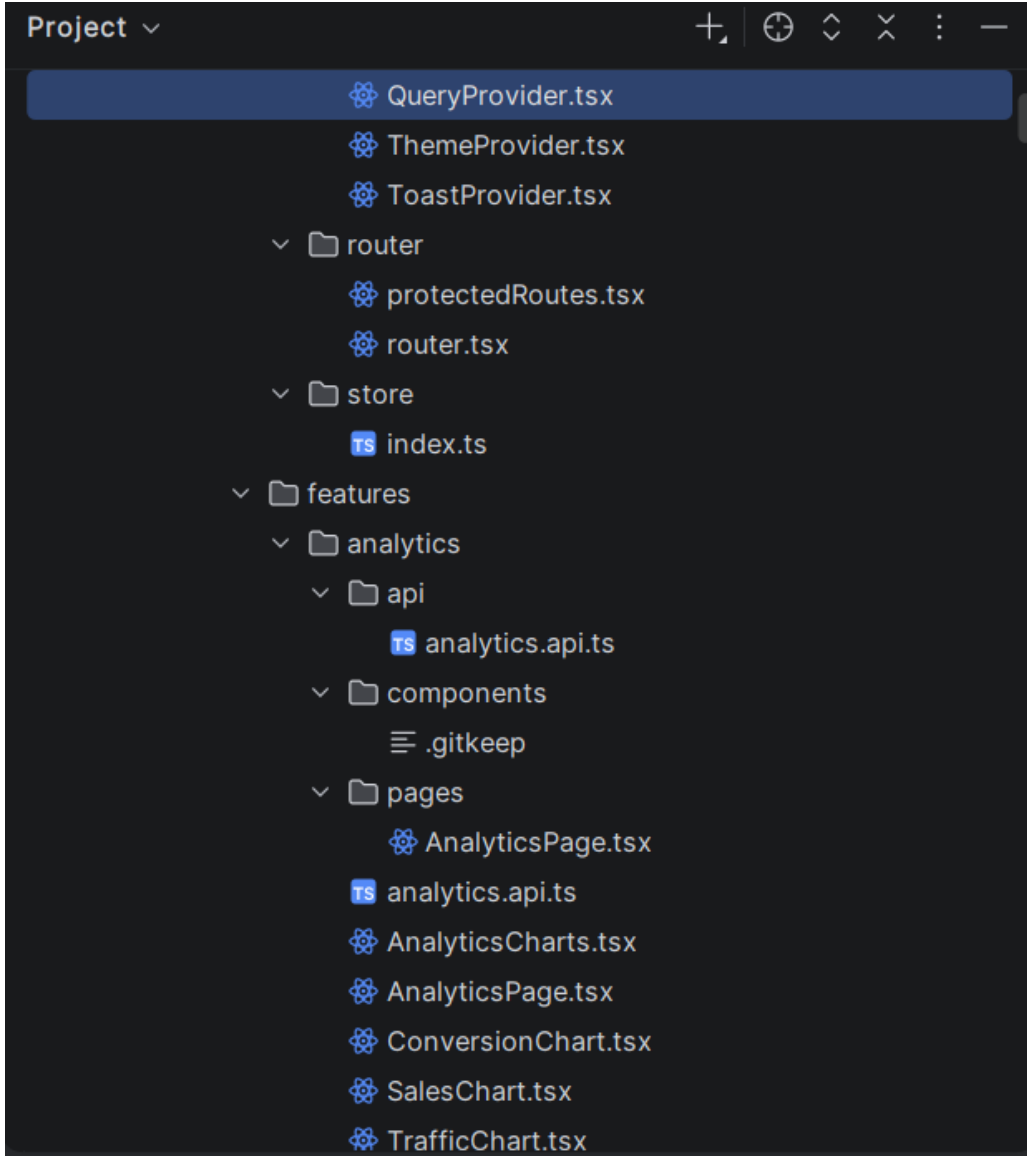
4.2. Teknoloji yığını ve geliştirme ortamı

4.2.1. Ortam ve ön yüz

Geliştirme süreci, sistem kararlılığını sağlamak adına Windows 10 ve 11 işletim sistemi ortamlarında yürütülmüştür. Uygulamanın çalışma zamanı gereksinimleri çerçevesinde, Node.js sürümü minimum v20 olarak sınırlandırılarak modern JavaScript özelliklerinden ve performans iyileştirmelerinden yararlanılmıştır. Proje mimarisi, monorepo yaklaşımını benimseyerek kök dizinde yer alan yapılandırma dosyası üzerinden npm workspaces aracılığıyla birden fazla uygulamayı tek bir çatı altında toplamaktadır. Bu yöntem, bağımlılık yönetimini merkezileştirirken farklı servisler arasındaki kod paylaşımını ve sürdürülebilirliği optimize etmektedir.

Ön yüz mimarisinde, yüksek performanslı ve tip güvenli bir geliştirme deneyimi sunan React 19, TypeScript ve Vite 6 teknolojileri tercih edilmiştir. Uygulama içi sayfa geçişleri ve rota yönetimi React Router kütüphanesi ile yapılandırılmıştır; bu noktada

müşteri tarafındaki son kullanıcı deneyimi için geleneksel tarayıcı yönlendiricileri kullanılırken, daha karmaşık veri akışına sahip olan satıcı panelinde createBrowserRouter API'si ile gelişmiş rota tanımlamaları yoluna gidilmiştir. Görsel tasarım ve arayüz tutarlılığı için Tailwind CSS 4 framework'ü kullanılarak modern ve duyarlı bir tasarım dili oluşturulmuştur. Kullanıcı etkileşimlerini güçlendirmek ve anlık sistem bildirimlerini yönetmek amacıyla react-toastify ve SweetAlert2 gibi yardımcı kütüphaneler entegre edilmiştir. Platformu oluşturan storefront, admin ve vendor-panel uygulamaları, frontend monorepo yapısı altındaki apps dizininde modüler bir biçimde konumlandırılmıştır. Bu ayrık yapı, her bir uygulamanın kendi iş mantığını korumasını sağlarken ortak bileşenlerin ve konfigürasyonların verimli bir şekilde yönetilmesine imkan tanımaktadır.



Şekil 4.2.1. Ön yüz dosyası

4.2.2. Arka uç ve altyapı

Sistemin arka uç mimarisi, yüksek ölçeklenebilirlik ve modülerlik hedefleri doğrultusunda Express 4 çatısı üzerine inşa edilmiştir. Veri erişim katmanında, veritabanı şeması ile uygulama kodu arasında tip güvenli bir köprü kuran Prisma ORM tercih edilmiş; üretim ortamında veri sürekliliği ve ilişkisel veri yönetimi için PostgreSQL veritabanı konumlandırılmıştır. Güvenlik ve kimlik yönetimi süreçlerinde, kullanıcı oturumlarının stateless bir yapıda takibi için jsonwebtoken kütüphanesi kullanılırken, hassas verilerin ve parolaların kriptografik olarak güvenli bir şekilde saklanması amacıyla bcryptjs algoritmaları entegre edilmiştir.

Uygulamanın ağ güvenliği ve veri işleme yetkinliklerini artırmak adına, çapraz kaynaklı istek yönetimi için cors, HTTP çerezlerinin ayrıştırılması için ise cookie-parser modülleri devreye alınmıştır. Sistemin web tabanlı saldırılara karşı direncini artırmak amacıyla Helmet ara yazılımı ile güvenlik başlıkları (security headers) yapılandırılmış; ayrıca servis dışı bırakma (DoS) saldırılarını ve aşırı kaynak tüketimini engellemek için istek gövdesi boyut sınırlandırmaları ile express-rate-limit mekanizmaları sisteme dahil edilmiştir. Medya yönetimi süreçlerinde, istemciden gelen dosya transferleri Multer ara yazılımı ile karşılanmakta, alınan görseller Sharp kütüphanesi aracılığıyla optimize edilip işlendikten sonra nesne depolama birimi olan MinIO üzerine aktarılmaktadır.

Performans optimizasyonu ve veri önbelleğe alma işlemleri için yüksek performanslı bir Redis istemcisi olan ioredis seçilmiştir. Sistemin izlenebilirliğini (observability) sağlamak adına operasyonel kayıtlar Winston kütüphanesi üzerinden günlüklenmekte ve bu verilerin merkezi bir panelden analizi için Seq entegrasyonu mimariye dahil edilmiştir. Kod organizasyonu tarafında ise sürdürülebilirlik ilkesi benimsenerek API yapılandırması src/api dizini altında etki alanlarına (domain) göre ayrıştırılmıştır. Her bir işlevsel modül, sorumlulukların ayrılması prensibi gereği kendi rotalarını (routes) ve iş mantığını barındıran kontrolcülerini (controllers) içerecek şekilde eşli yapılar halinde kurgulanmıştır.

4.2.3. Mikroservis iskeletleri

services klasöründeki auth-service, product-service ve order-service uygulamaları kendi Prisma şemaları ve Express giriş noktalarıyla ayrı süreçler gibi düşünülebilir.

Özellikle sipariş servisinde RabbitMQ'ya mesaj göndermeye yarayan yardımcı kod bulunur. Bu tezde anlattığım canlı senaryo ise tek birleşik API üzerinden gittiği için detaylar storebackend ile sınırlı tutulmuştur.

4.3. Mimari ve veri modeli

4.3.1. Mimari ve veri modeli

Proje dizininde yer alan services klasörü altındaki auth-service, product-service ve order-service bileşenleri, her biri kendi bağımsız Prisma şemalarına ve Express giriş noktalarına sahip müstakil servis yapıları olarak kurgulanmıştır. Bu mimari tercih, sistemin yatayda ölçeklenebilirliğini destekleyen ve servisler arası bağımlılığı minimize eden bir yaklaşımı temsil eder. Özellikle order-service modülü bünyesinde, asenkron veri iletişimi ve sistemler arası mesajlaşma protokollerini yönetmek amacıyla RabbitMQ mesaj kuyruğu sistemine veri aktarımı sağlayan yardımcı fonksiyonlar ve entegrasyon katmanları barındırılmaktadır.

Buna karşın, işbu tez çalışması kapsamında analiz edilen ve canlıya alınan senaryo, operasyonel bütünlüğün korunması ve karmaşıklığın yönetilmesi adına birleşik bir API mimarisi üzerinden yürütülmüştür. Bu doğrultuda, teknik çözümleme ve uygulama detayları, tüm bu servislerin temel işlevlerini bünyesinde barındıran ve sistemin ana operasyon merkezi konumunda olan storebackend uygulaması ile sınırlandırılmıştır. Uygulanan bu yöntem, veri tutarlılığının tek bir merkezden denetlenmesine olanak tanırken, akademik çalışmanın odak noktasını ana iş akışları üzerinde yoğunlaştırmaktadır.

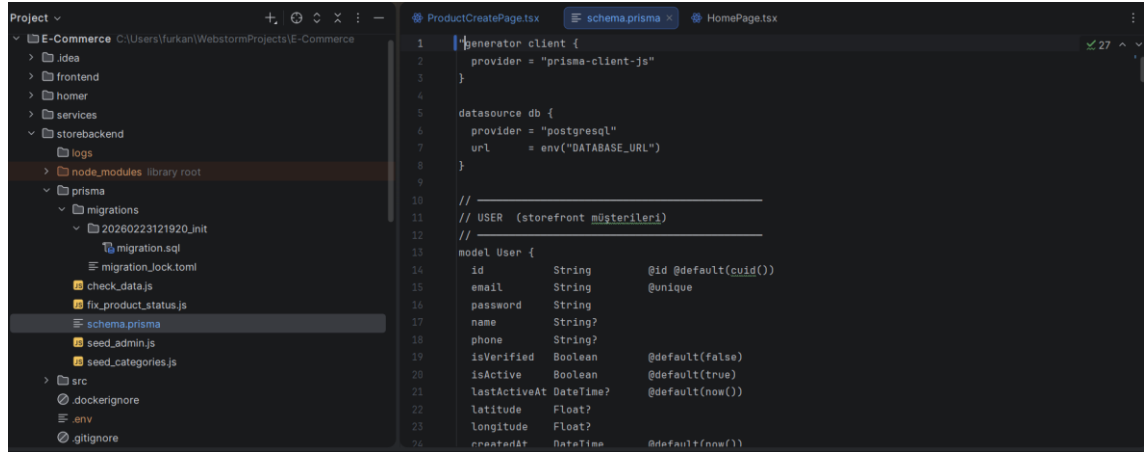
4.3.2. Veritabanı varlıkları

Sistemin tüm ilişkisel veri modeli, merkezi bir yapılandırma dosyası olan storebackend/prisma/schema.prisma üzerinde tanımlanmıştır. Kullanıcı hiyerarşisi; son kullanıcılar için User, ticari faaliyet yürüten paydaşlar için Merchant ve sistem denetleyicileri için Admin tabloları aracılığıyla ayrıştırılmıştır. Yönetici yetkilendirme süreçleri, AdminRole isimli sabit listesi (enum) ile standardize edilerek farklı erişim seviyeleri tanımlanmıştır. Oturum sürekliliğini ve güvenliğini sağlamak amacıyla, her üç kullanıcı tipi için (User, Merchant, Admin) bağımsız "Refresh Token" tabloları yapılandırılmış; bu sayede kimlik doğrulama süreçlerinde izolasyon ve izlenebilirlik maksimize edilmiştir.

Katalog yönetimi katmanında; hiyerarşik bir ağaç yapısı sunan Category tablosu, hem satıcıya hem de ilgili kategoriye lojik olarak bağlı Product tablosu ile ilişkilendirilmiştir. Ürün veri modeli; çoklu görsel desteği, ürün varyasyonları, dinamik ve filtrelenebilir bir Attribute (özellik) yapısı ve müşteri geri bildirimlerini içeren Review tablosu ile zenginleştirilmiştir. Kategori tanımlamalarında yer alan filterValues alanı, JSON formatında saklanarak ön yüzdeki vitrin filtreleme mekanizmalarına esnek ve performanslı bir veri kaynağı sunmaktadır.

Sipariş döngüsü, Order ve bu siparişe ait alt kalemleri temsil eden OrderItem modelleri üzerinden kurgulanmıştır. Her bir sipariş kaleminde (OrderItem) ilgili satıcıyı doğrudan işaret eden merchantId alanının bulunması, raporlama modüllerinde ve satıcı paneli üzerindeki filtreleme operasyonlarında veritabanı seviyesinde yüksek performanslı sorgulamalara imkan tanımaktadır. Ayrıca, sistem genelindeki operasyonel hareketlerin ve kritik işlemlerin takibi amacıyla SystemLog modeli mimariye dahil edilerek denetim izi (audit trail) mekanizması tesis edilmiştir.

Veri modelinin şematik yapısı, sistem bileşenleri arasındaki ilişkisel bütünlüğü ve veri akış yönlerini şu şekilde özetlemektedir:



Şekil 4.3.2. Veritabanı şeması ve prisma dosyası

4.3.3. Tutarlılık ve normalizasyon

Siparişin oluşturulma aşamasında, teslimat adresi verileri ilişkisel bir tablodan çekilmek yerine, işlemin gerçekleştiği andaki haliyle JSON formatında doğrudan sipariş kaydına (embedded) dahil edilmektedir. Benzer şekilde, sipariş kalemlerindeki birim fiyatlar da ürünün o anki güncel satış değerinden kopyalanarak sipariş satırına işlenmektedir. Bu veri mimarisi tercihi, ilerleyen süreçlerde müşteri profil bilgilerinde

veya ürün birim fiyatlarında yapılabilecek güncellemelerin, geçmişe dönük sipariş verilerini etkilemesini engellemektedir. Böylece, sipariş kayıtlarının hem hukuki geçerliliği hem de muhasebe standartları açısından veri tutarlılığı ve değişmezliği (immutability) garanti altına alınmıştır.

Sistemin katalog yapısında kategori ve ürün arasındaki ilişkisel modelleme, veritabanı normalizasyon kurallarına (Normal Form) uygun şekilde yapılandırılmıştır. Veri bütünlüğünü korumak ve mükerrer kayıt oluşumunu engellemek amacıyla, her bir ürün ve kategori için tanımlanan slug (URL uyumlu benzersiz isim) ve sku (stok tutma birimi) alanları üzerinde veritabanı seviyesinde benzersizlik (unique constraint) kısıtları uygulanmıştır. Bu kısıtlar, sistemin arama motoru optimizasyonu (SEO) süreçlerini desteklerken, envanter yönetiminde oluşabilecek operasyonel hataların önüne geçilmesini sağlamaktadır.

4.4. Sunucu tarafı uygulama

4.4.1. HTTP sunucusu ve güvenlik orta katmanları

Uygulamanın giriş noktası olan server.js dosyası, sistemin çalışma zamanı yapılandırmasını yönetmekte ve HTTP istek yaşam döngüsünü belirli bir hiyerarşiyle düzenleyen ara yazılım (middleware) katmanlarını ardışık olarak devreye almaktadır. Yapılandırma süreci; sistem güvenliğini pekiştiren Helmet başlıklarının tanımlanmasıyla başlar, ardından cookie-parser ile çerez yönetimi ve CORS politikalarının uygulanmasıyla devam eder. Veri iletim güvenliğini sağlamak adına istek gövdesi boyut sınırlandırmaları (payload limit) ve girdi temizleme (sanitization) adımları gerçekleştirilerek sisteme sızma girişimlerine karşı bir savunma hattı oluşturulur.

Uygulama katmanında, /api dizini altındaki tüm uç noktalar için özelleşmiş hız sınırlayıcılar (rate limiter) ve operasyonel izlenebilirliği sağlayan istek günlüğü (request logging) mekanizmaları aktif edilmiştir. Hata yönetimi stratejisi kapsamında, sistem genelinde meydana gelebilecek beklenmeyen istisnalar (exceptions) merkezi bir hata yakalama biriminde konsolide edilmektedir. Bu mimari yapı sayesinde, kaynak erişim politikaları (resource policy) nedeniyle reddedilen talepler ile sunucu tarafında oluşan teknik hatalar ayrıştırılmakta ve son kullanıcıya standartlara uygun, açıklayıcı farklı HTTP durum kodları ile geri bildirim sağlanmaktadır. Bu yaklaşım, hem sistemin hata toleransını artırmakta hem de istemci tarafındaki hata ayıklama süreçlerini kolaylaştırmaktadır.

4.4.2. API modülleri

Sistemin uygulama programlama arayüzü (API) mimarisi, farklı kullanıcı rolleri ve işlevsel modüller arasında net bir ayırım gözetilerek hiyerarşik bir uç nokta (endpoint) yapısı üzerine inşa edilmiştir. Kimlik doğrulama ve yetkilendirme süreçlerini yöneten servisler, güvenlik protokollerinin izolasyonu amacıyla üç temel örnek altında yapılandırılmıştır: son kullanıcılar için /api/auth/user, ticari paydaşlar için /api/auth/merchant ve sistem denetleyicileri için /api/auth/admin. Katalog yönetimi ve ürün yaşam döngüsü operasyonları; temel ürün verileri için /api/products, meta veriler ve teknik spesifikasyonlar için /api/products-meta, hiyerarşik yapılandırmalar için /api/categories ve dinamik filtreleme bileşenleri için /api/attributes yolları üzerinden yürütülmektedir.

Alişveriş deneyiminin merkezinde yer alan sepet modülü, /api/cart uç noktası altında konumlandırılmış olup; hem anonim oturumları hem de kimliği doğrulanmış kullanıcı oturumlarını destekleyecek esnek bir mantıksal yapı ile kurgulanmıştır. Kullanıcı bağlılığını artırmaya yönelik "istek listesi" ve "tematik koleksiyon" işlevleri sırasıyla /api/wishlist ve /api/collections yolları üzerinden sunulmaktadır. Operasyonel veri akışının kritik bileşenleri olan sipariş yönetimi ve müşteri geri bildirimleri, /api/orders ve /api/reviews servisleri aracılığıyla birbirinden ayrıştırılmıştır. Platformun idari ve ticari yönetim katmanları ise rol tabanlı erişim kontrolü (RBAC) prensipleri gereği /api/merchant ve /api/admin örnekleri altında konsolide edilmiştir. Son olarak, sistem genelindeki tüm medya ve doküman transfer süreçleri, merkezi bir dosya yükleme servisi olan /api/upload uç noktası ile sonlandırılmaktadır.

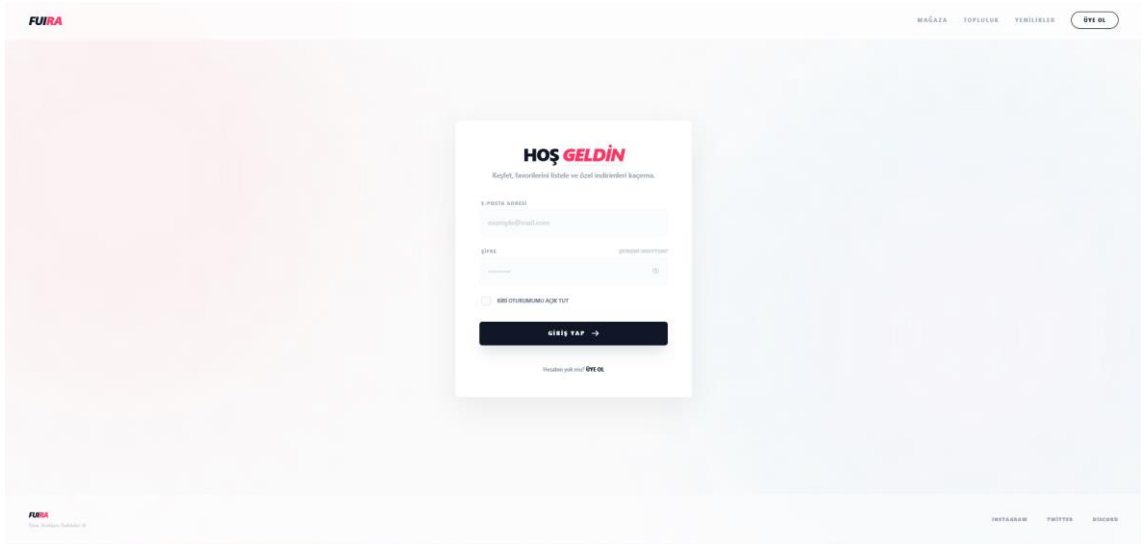
4.4.3. Kimlik doğrulama ve yetkilendirme

Sistem güvenliğinin temel taşlarından biri olan kimlik bilgileri yönetimi kapsamında, kullanıcı parolaları veritabanı seviyesinde hiçbir aşamada açık metin (plain text) formatında saklanmamakta; bunun yerine bcrypt algoritması kullanılarak tek yönlü kriptografik özetleme (hashing) işlemine tabi tutulmaktadır. Kimlik doğrulama mimarisi, sistem sürdürülebilirliğini ve güvenliğini optimize etmek amacıyla kısa ömürlü erişim jetonları (access tokens) ve uzun ömürlü yenileme jetonları (refresh tokens) kombinasyonu üzerine kurgulanmıştır.

Geliştirilen authenticate ara yazılımı (middleware), gelen istekleri doğrularken öncelikle HTTP Authorization başlığındaki "Bearer" jetonunu kontrol etmekte; bu

verinin eksikliği durumunda ise sistem rolleriyle ilişkilendirilmiş ve tarayıcı tarafından erişilemeyen HttpOnly çerezlerine (cookies) yönelmektedir. Doğrulama süreci verifyAccessToken fonksiyonu ile yürütülmekte ve başarılı olan kimlik doğrulama yükleri (payload), istek nesnesi içerisindeki req.user alanına aktarılarak iş mantığı katmanına iletilmektedir. Yetkilendirme düzeyinde ise veri izolasyonunu sağlamak amacıyla, örneğin bir satıcının yalnızca kendi envanterindeki ürünleri güncelleyebilmesi gibi kısıtlar, kontrolcü (controller) katmanında merchantId karşılaştırmaları ve mülkiyet doğrulamaları ile güvence altına alınmaktadır.

Müşteri arayüzü tarafında, oturum yönetimi authStore sınıfı aracılığıyla koordine edilmekte ve kullanıcı profil bilgileri tarayıcı yerel depolama biriminde (localStorage) muhafaza edilmektedir. Ancak, erişim jetonları güvenlik gereği HttpOnly çerez yapısında saklandığı için istemci tarafındaki JavaScript kodu, bu jetonları doğrudan okuma yetkisine sahip olmayacak şekilde konfigüre edilmiştir; bu yaklaşım, Cross-Site Scripting (XSS) saldırılarına karşı kritik bir koruma kalkanı oluşturmaktadır. Oturum sonlandırma (logout) işlemlerinde yerel depolamadaki profil verileri temizlenmekte, sunucu tarafında ise ilgili yenileme jetonunun iptal edilmesi (revocation) ve çerezlerin geçersiz kılınması sağlanarak oturum tam anlamıyla sonlandırılmaktadır. Bu çift katmanlı oturum kapatma mekanizması, hem istemci hem de sunucu tarafında tam veri senkronizasyonu ve işlem güvenliği tesis etmektedir.



Şekil 4.4.3 Giriş sayfası

4.4.4. Sipariş oluşturma ve atomik işlem

Sipariş oluşturma süreci, veri bütünlüğünü ve operasyonel tutarlılığı en üst düzeyde tutmak amacıyla Prisma ORM tarafından sunulan \$transaction bloğu içerisinde, atomik bir işlem olarak yürütülmektedir. Süreç, istemciden gelen istek gövdesinin doğrulanmasıyla başlar; bu aşamada her bir sipariş kalemi için ilgili ürün verisi veritabanından tekrar okunur. Toplam sipariş tutarı hesaplanırken, istemci tarafından gönderilen birim fiyatlar yerine sunucu tarafındaki güncel ve doğrulanmış satış fiyatları baz alınır. Bu yöntem, fiyat manipülasyonu gibi güvenlik risklerini ortadan kaldırarak finansal doğruluğu garanti altına alır.

İşlem bloğu içerisinde sipariş üst verileri ve sipariş kalemleri eş zamanlı olarak veritabanına işlenirken, ilgili ürünlerin stok sayaçları da gerçek zamanlı olarak düşürülür. İşlemlerin tek bir transaksyon altında toplanması, siparişin kaydedilip stok güncellemesinin başarısız olması gibi senaryoları engellemekte; yani işlemlerin ya tamamen gerçekleşmesini ya da bir hata durumunda tamamen geri alınmasını (rollback) sağlamaktadır. Geliştirme safhasında ödeme durumu doğrudan onaylanmış kabul edilecek şekilde yapılandırılabilse de, üretim ortamında bu alanın güncellenmesi entegre edilen harici ödeme geçitleri (payment gateways) aracılığıyla gerçekleştirilmelidir. Yüksek hacimli ve eş zamanlı satış taleplerinin (race conditions) söz konusu olduğu senaryolarda stok çakışmalarını önlemek amacıyla, veritabanı yalıtım düzeyleri (isolation levels) veya iyimser/kötümser kilitleme (optimistic/pessimistic locking) stratejileri mimariye dahil edilerek veri tutarlılığı korunmaktadır.

Şekil 4.4.4 Sipariş oluşturma sayfası

4.4.5. Redis, koleksiyonlar ve kategori filtreleri

Kullanıcıya özgü oluşturulan koleksiyon listeleri ve tercih verileri, yüksek performanslı erişim sağlamak amacıyla Redis bellek içi veri yapısında anahtar-değer (key-value) formatında muhafaza edilmektedir. Her bir koleksiyon kaydı için oluşturulan anahtar deseni (key pattern), kullanıcı kimliği (UID) ile benzersiz şekilde ilişkilendirilerek veri çakışmalarının önüne geçilir. Bellek yönetimini optimize etmek ve güncelliğini yitirmiş verilerin sistem kaynaklarını işgal etmesini engellemek amacıyla, her bir kayıt için yaklaşık doksan günlük bir geçerlilik süresi (TTL - Time To Live) tanımlanmıştır.

Platformun vitrin katmanında sunulan dinamik filtreleme mekanizmaları için /api/products-meta uç noktaları operasyonel rol oynamaktadır. Kullanıcı tarafından seçilen kategoriye ait "slug" değeri üzerinden başlatılan işlem sürecinde, ilgili kategorinin alt dallarındaki tüm hiyerarşik kimlikler bellek üzerinde taranarak kapsamlı bir ürün kümesi oluşturulur. Bu süreçte, hiyerarşinin tepesinde yer alan kök kategorinin filterValues alanında tanımlanmış olan yapılandırma verileri, arayüzdeki filtre bileşenlerini (fiyat aralığı, marka, teknik özellikler vb.) dinamik olarak beslemektedir. Bu

mimari yaklaşım, hem veritabanı üzerindeki sorgu yükünü hafifletmekte hem de son kullanıcıya kategori bazlı özelleştirilmiş ve hızlı bir arama deneyimi sunmaktadır.

Model, özellik veya satıcı...

KATEGORİ

HEPSİ

KİTAP, MÜZİK, HOBİ

ANNE & BEBEK

SÜPERMARKET

KİZ

SDG

ELEKTRONİK

MODA

EV & YAŞAM

FİYAT ARALIĞI

MIN 899,9

MAX 124999

SATICILAR

FİLTRELERİ SIFIRLA

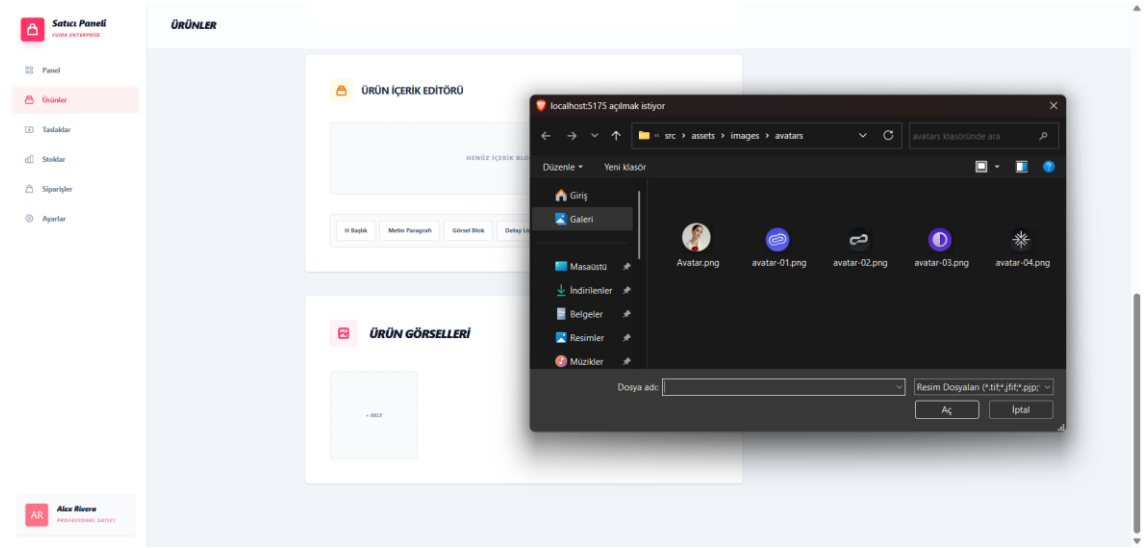
Şekil 4.4.5. Kategori filtreme bileşeni

4.4.6. Dosya yükleme ve medya

Dosya yönetim sistemi, sunucu kaynaklarını optimize etmek ve veritabanı performansını korumak amacıyla katmanlı bir yapıda kurgulanmıştır. İstemci tarafından gönderilen medya içerikleri ve dökümanlar, öncelikle Multer ara yazılımı (middleware) aracılığıyla sunucu tarafında karşılanmaktadır. Alınan ham görüntü verileri, bellek ve depolama verimliliğini artırmak amacıyla Sharp kütüphanesi kullanılarak işleme tabi

tutulur; bu aşamada yeniden boyutlandırma, format dönüştürme ve sıkıştırma gibi optimizasyon işlemleri gerçekleştirilir.

İşlenen nihai dosyalar, yapılandırılmamış veriler için yüksek ölçeklenebilirlik sunan nesne depolama (object storage) çözümü MinIO üzerine aktarılmaktadır. Veritabanı katmanında ise veri şişmesini (database bloating) önlemek adına, büyük boyutlu ikili dosyaların (binary large objects - BLOB) doğrudan tablolarda saklanması yerine, yalnızca ilgili nesneye erişim sağlayan benzersiz anahtar (key) veya URL adresi tutulmaktadır. Bu mimari yaklaşım, ilişkisel veritabanının sorgu hızını ve yedekleme süreçlerini stabilize ederken, medya içeriklerinin bağımsız bir depolama birimi üzerinden yüksek bant genişliğiyle sunulmasına olanak tanımaktadır.



Şekil 4.4.6 Görsel yükleme bileşeni

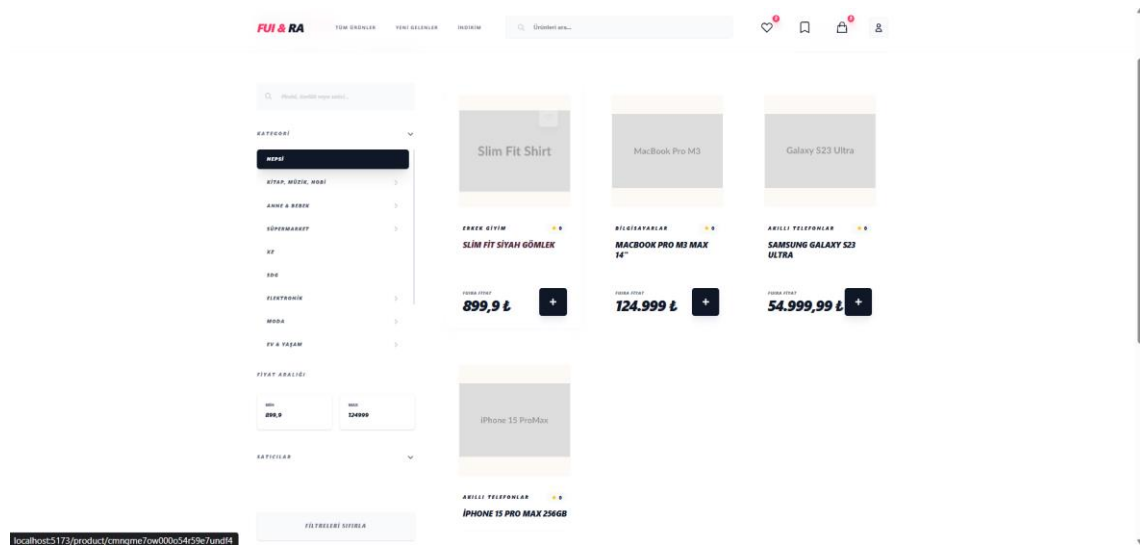
4.5. İstemci uygulamaları

4.5.1. Mağaza

Müşteri arayüzü yönlendirme mimarisi, kullanıcı deneyimini optimize etmek amacıyla modüler bir rota yapısı üzerine kurgulanmıştır. Platformun ana giriş noktası ve tematik vitrin varyasyonları /, /shop, /new ve /sale dinamik uç noktaları üzerinden sunulmaktadır. Ürün hiyerarşisinde derinlemesine inceleme imkanı sağlayan ürün detay sayfaları, ilgili içeriğe özgü benzersiz kimlik parametreleri (ID veya slug) aracılığıyla veri çekmektedir.

Alışveriş döngüsünün operasyonel bileşenleri olan sepet yönetimi, ödeme öncesi doğrulama adımları, kullanıcı profili, geçmiş sipariş dökümleri, istek listesi ve kişiselleştirilmiş koleksiyonlar için birbirinden bağımsız ve semantik rota tanımlamaları

yapılmıştır. Güvenlik ve erişim yönetimi kapsamında, kimlik doğrulama ve yeni kayıt sayfaları genel uygulama akışından izole edilerek farklı bir düzen (layout) içerisinde yapılandırılmıştır. Uygulama genelinde görsel tutarlılığı sağlamak ve kod tekrarını minimize etmek amacıyla sayfa iskeleti MainLayout bileşeni ile standartlaştırılmıştır. Form girişleri, veri gönderim süreçleri ve sistem operasyonlarının sonuçları, kullanıcıya anlık geri bildirim sağlayan interaktif toast bildirimleri ve modal diyalog pencereleri aracılığıyla raporlanmaktadır.



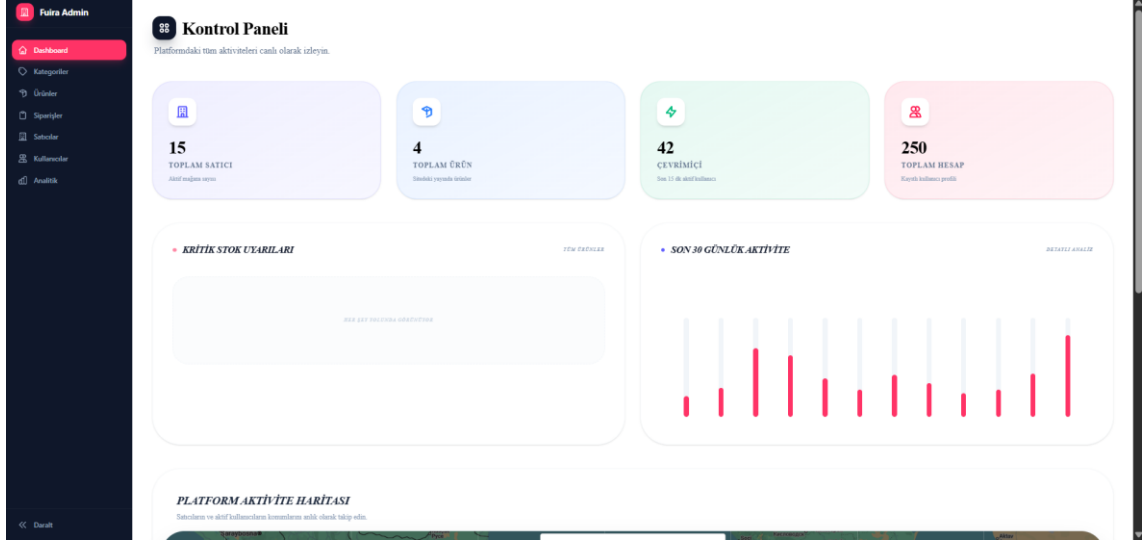
Şekil 4.5.1 Mağaza sayfası

4.5.2. Yönetici paneli

Platform yönetim arayüzü, sistem genelindeki operasyonel süreçlerin denetimi ve veri analitiği için tasarlanmış merkezi bir kontrol paneli (dashboard) hiyerarşisi üzerine inşa edilmiştir. İdari kullanıcılar için sunulan rota yapısı; kategorilerin hiyerarşik düzenlenmesi, ürün özellik sözlüklerinin (attributes) yönetimi, kapsamlı ürün ve sipariş listeleme modülleri ile satıcı ve son kullanıcı veri tabanlarının denetimini kapsayan özelleşmiş uç noktalarından oluşmaktadır. Ayrıca, platformun ticari performansını izlemek amacıyla hazırlanan analitik veri ekranları da bu rota ağacına entegre edilmiştir.

Güvenlik ve tutarlılık prensipleri gereği, tüm yönetim işlevleri yetkilendirme katmanı (RBAC) ile korunmakta ve yalnızca kimlik doğrulaması yapılmış idari personelin erişimine açık, izole bir çalışma alanında sunulmaktadır. Arayüz tasarımı, operasyonel verimliliği artırmak ve kullanıcı deneyimini standartlaştırmak adına "MainLayout" benzeri tek tip bir mimari düzen (layout) içerisinde yapılandırılmıştır. Bu

modüler yapı, karmaşık veri setlerinin yönetilmesini kolaylaştırırken, farklı yönetim fonksiyonları arasında akıcı bir geçiş imkanı sağlamaktadır.

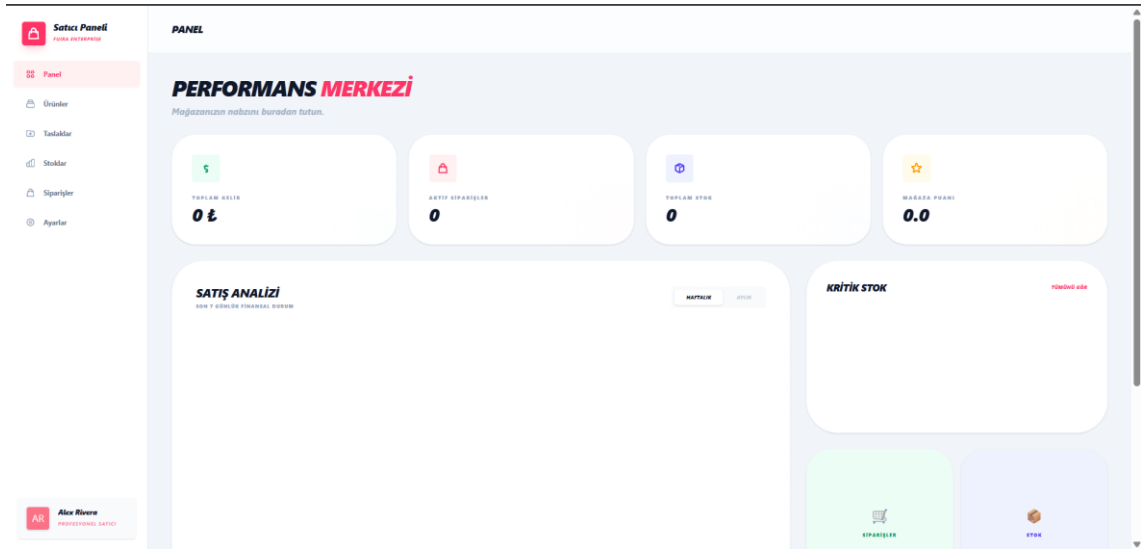


Şekil 4.5.2. Yönetici paneli

4.5.3. Satıcı paneli

Satıcı paneli mimarisi, ticari operasyonların uçtan uca yönetilebilmesi amacıyla fonksiyonel modüllere ayrıştırılmıştır. Uygulama; genel performans verilerinin izlendiği merkezi pano (dashboard), mevcut envanterin yönetildiği ürün listeleme ekranı, yeni veri girişi ve güncelleme işlemlerini kapsayan oluşturma/düzenleme modülleri, henüz tamamlanmamış çalışmalar için ayrıştırılmış taslak listesi, sipariş takip birimi, gerçek zamanlı stok izleme ekranı ve kurumsal konfigürasyonların yönetildiği ayarlar sayfasından teşekkül etmektedir.

Sistem güvenliği ve veri izolasyonu prensipleri gereği, yetkilendirilmemiş erişimleri engellemek amacıyla tüm hassas rotalar ProtectedRoute ara yazılımı (high-order component) ile koruma altına alınmıştır; bu mekanizma, geçerli bir oturumun bulunmadığı durumlarda kullanıcıyı otomatik olarak kimlik doğrulama sayfasına yönlendirmektedir. Kullanıcı deneyimini kesintisiz kılmak ve veri kaybını önlemek adına, ağ bağlantısı durumunu anlık olarak izleyen ve bağlantı kesintilerinde kullanıcıya görsel uyarılar sunan OfflineDetector bileşeni mimariye entegre edilmiştir. Ürün yaşam döngüsünün takibi ise veri tutarlılığını sağlamak amacıyla ProductStatus sabit listesi (enum) üzerinden yürütülmekte; bu kapsamda bir ürünün sistemdeki mevcudiyeti "taslak", "yayında" veya "arşivlenmiş" durum kodlarıyla dinamik olarak izlenmektedir.



Şekil 4.5.3 Satıcı paneli

4.5.4. Kullanıcı deneyimi notları

Tek bir vitrin bileşeninin birden fazla dinamik URL yapısına hizmet verecek şekilde kurgulanması, yazılım geliştirme süreçlerinde kod tekrarını (DRY - Don't Repeat Yourself) minimize eden ve bakım maliyetlerini düşüren stratejik bir yaklaşımdır. Bu mimari yapı, aynı görsel arayüzün ve veri listeleme mantığının farklı semantik bağlamlarda (örneğin kampanya sayfaları, kategori bazlı filtrelemeler veya yeni gelen ürünler) yeniden kullanılabilmesine olanak tanır. Ancak, sistemin sürdürülebilirliği ve arama motoru optimizasyonu (SEO) başarısı açısından, hangi URL yapısının hangi işlevsel amaca hizmet ettiğinin ve hangi veri setini temsil ettiğinin teknik dokümantasyon ile netleştirilmesi kritik öneme sahiptir.

Arama motoru botlarının ve içerik paylaşım protokollerinin (Open Graph vb.) sistemi doğru anlamlandırması için, çoklu adres yapılarında "canonical URL" tanımlamaları ve yönlendirme stratejileri titizlikle kurgulanmalıdır. Bu kapsamda, her bir adresin semantik anlamı; ana sayfa varyantları için /, genel katalog taraması için /shop, yeni eklenen ürün grupları için /new ve indirimli ürünlerin konsolide edildiği /sale gibi uç noktalar üzerinden teknik olarak tanımlanmalı ve bu adreslerin birbirleriyle olan ilişkisel farkları dökümanite edilmelidir. Uygulanan bu yöntem, hem tarayıcı tarafında rota yönetimini (routing) standartlaştırmakta hem de platformun dijital görünürlüğünü destekleyen yapılandırılmış bir veri hiyerarşisi sunmaktadır.

4.6. İş kuralları ve senaryolar

4.6.1. Sepet

Sistem mimarisinde sepet uç noktaları (endpoints), kullanıcı kimlik doğrulaması zorunluluğu gözetilmeksizin esnek bir yapıda kurgulanmaktadır. Bu yaklaşım, platformun dönüşüm oranlarını artırmak amacıyla anonim kullanıcıların (misafir) alışveriş deneyimine kesintisiz başlamasına olanak tanır. Ancak, misafir oturumu sırasında oluşturulan geçici sepet verileri ile kullanıcı sisteme giriş yaptıktan sonra mevcut olan kalıcı sepet verilerinin birleştirilmesine (cart merging) yönelik iş kurallarının teknik düzeyde netleştirilmesi kritik bir gerekliliktir.

Birleştirme stratejisi uygulanmadığı takdirde, kullanıcılar hesaplarına eriştiklerinde daha önce seçtikleri ürünlerin sepetten silindiği veya senkronize edilmediği bir senaryoyla karşılaşarak "veri kaybı" algısına kapılabilmektedir. Bu durum, müşteri memnuniyetini olumsuz etkileyen önemli bir teknik sürtünme kaynağıdır. Sistemin bu sorunu aşmak için izleyebileceği yöntemlerden biri, oturum açma (login) işlemi gerçekleştiği anda istemci tarafındaki (localStorage veya çerez tabanlı) misafir sepeti ile sunucu tarafındaki kullanıcı sepetini birleştiren bir asenkron senkronizasyon mekanizması devreye almaktır. Bu süreçte, aynı ürünlerin çakışması durumunda miktar güncellemesi yapılması veya en son eklenen verinin önceliklendirilmesi gibi mantıksal kurallar, veritabanı tutarlılığı ve kullanıcı deneyimi (UX) arasındaki dengeyi sağlamak adına sistem dökümantasyonunda tanımlanmalıdır.

4.6.2. Çok satıcılı pazar yeri

Çok satıcılı (multi-vendor) mimarinin temel direği olan veri izolasyonu, sistemde her satıcının yalnızca kendisine ait veri setine erişebilmesi prensibi üzerine kurgulanmıştır. Bu izolasyonun teknik uygulama yöntemi, veritabanı seviyesindeki tüm sorgularda merchantId parametresi üzerinden gerçekleştirilen dinamik filtreleme mekanizmasıdır. Tek bir müşteri siparişi içerisinde farklı satıcılara ait birden fazla ürün kalemi (order items) bulunabildiğinden, sistem mimarisi bu kalemleri satıcı bazlı ayıştıracak şekilde yapılandırılmıştır. Lojistik dağıtım süreçleri, hak ediş hesaplamaları ve komisyon yapılandırılmaları gibi ileri düzey ticari fonksiyonlar, ana sistemin modüler yapısını bozmadan ayrı iş mantığı katmanları (business logic layers) üzerinden derinleştirilmeye uygun bir esneklik sunmaktadır.

Satıcı paneli uygulaması, ticari operasyonların uçtan uca yönetilebilmesi amacıyla fonksiyonel bir kullanıcı arayüzü hiyerarşisi ile tasarlanmıştır. Uygulama; genel performans metriklerinin izlendiği merkezi pano (dashboard), mevcut envanterin yönetildiği ürün listeleme ekranı, yeni veri girişi ve güncelleme modülleri, henüz yayına alınmamış çalışmalar için ayrıştırılmış taslak listesi, sipariş takip birimi, gerçek zamanlı stok izleme ekranı ve kurumsal yapılandırmaların yönetildiği ayarlar sayfasından teşekkül etmektedir.

Güvenlik katmanında, yetkilendirilmemiş erişimleri engellemek amacıyla tüm hassas rotalar ProtectedRoute bileşeni ile koruma altına alınmış; geçerli bir oturumu bulunmayan kullanıcıların otomatik olarak kimlik doğrulama sayfasına yönlendirilmesi sağlanmıştır. Kullanıcı deneyimini kesintisiz kılmak ve veri kaybını önlemek adına, ağ bağlantı durumunu anlık olarak denetleyen ve olası kesintilerde kullanıcıyı teknik aksaklıklara karşı uyararak OfflineDetector bileşeni mimariye dahil edilmiştir. Ürünlerin yaşam döngüsü ise veri tutarlılığını korumak amacıyla ProductStatus sabit listesi (enum) üzerinden takip edilmekte; bu kapsamda her bir ürünün sistemdeki mevcudiyeti "taslak", "yayında" veya "arşivlenmiş" durum kodları ile dinamik olarak izlenmektedir.

4.6.3. Uçtan uca akış

Platformun temel iş akışı, son kullanıcı deneyimi ile arka uç operasyonel süreçlerinin entegrasyonu üzerine kurulu doğrusal bir rota izlemektedir. Bu sürecin başlangıç noktasını, vitrin katmanında gerçekleştirilen dinamik ürün keşfi ve navigasyon işlemleri oluşturur. Kullanıcı etkileşimi; yeni hesap oluşturma veya mevcut oturuma giriş yapma safhasını takiben, seçilen ürünlerin sepet mekanizmasına dahil edilmesi ve lojistik teslimat bilgilerinin sisteme girilerek siparişin onaylanmasıyla devam eder.

Teknik arka planda bu onaylama işlemi, veri tutarlılığını garanti altına almak amacıyla sunucu tarafında atomik bir veritabanı transaksyonu içerisinde yürütülür; bu aşamada sipariş kaydı oluşturulurken eş zamanlı olarak ilgili ürünlerin stok güncellemeleri gerçekleştirilir. Siparişin başarıyla sisteme işlenmesinin ardından süreç, çok yönlü bir izleme fazına evrilir: Son kullanıcı kendi profili üzerinden sipariş durumunu takip edebilirken, satıcı paydaşları kendilerine özgü panel üzerinden sipariş kalemlerini (order items) baz alarak operasyonel takibi sürdürür. Sistemin en üst katmanında ise, gerekli görülen durumlarda veri bütünlüğünü veya müşteri memnuniyetini sağlamak

adına yönetici müdahalesi ve denetim mekanizmaları devreye girerek uçtan uca döngüyü tamamlar.

4.6. İş kuralları ve senaryolar

4.7. Güvenlik, günlük ve hata yönetimi

Sistem güvenliği ve operasyonel sürdürülebilirlik, birbiriyle entegre çalışan çok katmanlı savunma ve izleme mekanizmaları üzerine inşa edilmiştir. Helmet ve CORS (Cross-Origin Resource Sharing) protokollerinin birlikte yapılandırılması, uygulamanın saldırı yüzeyini daraltarak tarayıcı tabanlı güvenlik açıklarını minimize eder. Sistemin kaynak tüketimini denetim altında tutmak ve servis dışı bırakma (DoS) girişimlerini engellemek amacıyla, istek gövdesi boyut sınırlandırmaları ve birim zamandaki istek sayısını kısıtlayan hız limitleri (rate limiting) devreye alınmıştır. Kimlik doğrulama ve oturum yönetimi süreçleri ise, kullanıcı hareketliliğini ve güvenliğini optimize eden kısa ömürlü erişim jetonları ile uzun ömürlü yenileme jetonları (JWT) aracılığıyla yürütülmektedir.

Sistemin izlenebilirliğini (observability) sağlamak adına, operasyonel kayıtlar Winston kütüphanesi üzerinden günlüklenmekte ve bu verilerin merkezi bir panelden analizi için isteğe bağlı olarak Seq entegrasyonu sunulmaktadır. Uygulama katmanında ise, kritik işlemlerin ve denetim izlerinin (audit trail) kalıcı olarak saklanması amacıyla SystemLog modeli kullanılmaktadır. Hata yönetimi stratejisi kapsamında, öngörülemeyen sonucu tarafı istisnaları (exceptions) merkezi bir hata yakalama biriminde konsolide edilmektedir. Bu mimari yaklaşım, sunucudan dönen başarı ve hata kodlarının standart bir yapıda tutulmasını sağlayarak, istemci tarafındaki arayüz kodunun karmaşıklığından arınmasına ve daha yalın bir yapıda kalmasına olanak tanımaktadır.

Kullanıcı güvenini tesis etmeye yönelik sosyal kanıt mekanizmaları kapsamında, ürün yorumları modülünde isVerified (doğrulanmış alıcı) gibi işaretleyiciler sisteme dahil edilmiştir. Bu veri alanları, yorumun gerçekten ürünü satın alan bir kullanıcı tarafından yapıldığını teyit ederek platformun güvenilirliğini pekiştirmeyi amaçlamaktadır. Platformun kullanıcı trafiği ve veri hacmi genişledikçe, içerik kalitesini korumak adına moderasyon süreçlerinin ve otomatik denetim mekanizmalarının sisteme entegrasyonu, operasyonel bir gereklilik olarak öngörülmektedir.

4.8. Monorepo, kod yapısı ve mimari perspektif

Sistemin mimari organizasyonu, sürdürülebilirlik ve modüler genişleme hedefleri doğrultusunda hem arka uçta hem de ön yüzde katmanlı bir yapı üzerine inşa edilmiştir. Arka uç katmanında, her bir iş alanı (domain) src/api dizini altında kendine ait bağımsız klasörlerde konumlandırılmıştır; bu yaklaşım, kod tabanındaki karmaşıklık azaltarak modüller arası bağımlılığı minimize eder. Ön yüz mimarisinde ise işlevsel yetkinlikler features klasörü altında toplanırken, uygulamanın genelinde kullanılan tekrar edilebilir bileşenler ve yardımcı fonksiyonlar shared dizini içerisinde konsolide edilmiştir. Her üç uygulama (storefront, admin, vendor) aynı RESTful protokol sözleşmesini paylaştığından, sistemin geriye dönük uyumluluğunu korumak ve yeni özellikleri güvenli bir şekilde devreye almak amacıyla gelecekte /api/v1 gibi versiyonlama örneklerinin (API versioning) kullanımı öngörülmektedir.

Geliştirme sürecinin başlangıcında tercih edilen tek parçalı storebackend mimarisi, operasyonel basitlik ve hızlı prototipleme avantajı sunmaktadır. Buna karşın, sistem hiyerarşisinde yer alan ayırık servis (microservices) yaklaşımı, dağıtım süreçleri ve veri tutarlılığı yönetimi açısından daha yüksek bir mühendislik maliyeti gerektirse de, sistem ölçeklendiğinde bileşenlerin birbirinden bağımsız olarak genişletilebilmesine olanak tanıyan esnek bir yapı vaat etmektedir. Bu bağlamda, asenkron işlem yönetimi için mimariye dahil edilebilecek RabbitMQ gibi mesaj kuyruğu sistemleri, sipariş sonrası bildirim gönderimi veya ağır veri işleme görevlerini ana API yanıt döngüsünden ayırarak sistemin genel tepki süresini (latency) optimize etme potansiyeline sahiptir. Bu modüler yapı, platformun büyüme evrelerinde ihtiyaç duyulacak teknik çevikliği ve performans sürekliliğini garanti altına almaktadır.

4.9. Performans, ölçek, risk ve test

Sistem performansını optimize etmek amacıyla, sık filtrelenen veri alanları üzerinde veritabanı indeksleme stratejileri uygulanmakta ve veri setlerinin sunumunda sayfalama (pagination) mekanizmaları devreye alınmaktadır. Statik içeriklerin dağıtımında içerik dağıtım ağları (CDN) kullanımı ve görsel materyallerde yüksek sıkıştırma oranına sahip WebP gibi modern formatların tercih edilmesi, yükleme sürelerini (latency) minimize eden kritik hamlelerdir. Platformun trafik hacmi genişledikçe, veritabanı üzerindeki okuma yükünü hafifletecek okuma replikaları (read

replicas) ve Redis tabanlı gelişmiş önbellekleme (caching) stratejilerinin mimariye dahil edilmesi operasyonel bir gereklilik olarak öngörülmektedir.

Sistem güvenliği ve sürekliliği açısından tanımlanan başlıca riskler; erişim jetonu sızıntıları, eş zamanlı siparişlerde oluşabilecek stok yarışları (race conditions), Redis servis kesintileri ve denetlenmeyen dosya yükleme girişimlerini kapsamaktadır. Bu risk vektörlerine karşı hem uygulama kodu seviyesinde hem de altyapı katmanında özelleşmiş karşı önlemler (countermeasures) tesis edilmiştir. Yazılım kalitesini korumak ve teknik borç birikimini engellemek amacıyla, otomatik test kapsamının artırılması stratejik bir öneme sahiptir. Bu doğrultuda; kritik sipariş akışları ve yetkilendirme katmanları için Prisma tabanlı veritabanı testleri, Supertest kütüphanesi ile HTTP uç nokta testleri ve React Testing Library aracılığıyla arayüz bileşen testlerinin entegrasyonu, sistemin uzun vadeli kararlılığını garanti altına alacak bir kalite güvence (QA) çerçevesi sunmaktadır.

4.10. Derleme, Docker ve dağıtım

4.10.1. Üretim derlemesi

Vite derleme süreci, geliştirme aşamasındaki kaynak kodları üretim ortamına hazır, optimize edilmiş statik varlıklara (bundle) dönüştürmek amacıyla vite build komutu üzerinden yürütülür. Bu aşamada JavaScript, TypeScript ve CSS dosyaları minize edilerek tarayıcı tarafındaki yükleme performansını maksimize edecek şekilde paketlenir. Uygulamanın farklı çalışma ortamlarında (geliştirme, test, üretim) esnek bir yapıda çalışabilmesi için, API uç noktaları ve diğer yapılandırma parametreleri sisteme sabit kodlanmak yerine ortam değişkenleri (environment variables) aracılığıyla enjekte edilir.

Vite mimarisi gereği, istemci tarafındaki kodun erişebileceği tüm değişkenlerin VITE_ öneki ile tanımlanması zorunluluğu bulunmaktadır; bu güvenlik mekanizması, sunucu tarafındaki hassas verilerin yanlışlıkla istemci paketine dahil edilmesini engeller. .env dosyaları üzerinden yönetilen bu yapılandırma stratejisi, uygulamanın kod tabanını değiştirmeye gerek kalmaksızın, yalnızca ortam değişkenlerini güncelleyerek farklı sunucu altyapılarına kolayca uyarlanabilmesine olanak tanımaktadır. Bu yöntem, sürekli entegrasyon ve sürekli dağıtım (CI/CD) süreçlerinde operasyonel çeviklik sağlayarak modern yazılım geliştirme standartlarını desteklemektedir.

4.10.2. Docker Compose

Projenin kök dizininde yer alan docker-compose.yml yapılandırma dosyası; ilişkisel veritabanı, Redis önbellekleme servisi, Seq günlükleme platformu, MinIO nesne depolama birimi, ana arka uç servisi (storebackend) ve üç farklı ön yüz uygulama konteynerinin tek bir komutla orkestre edilmesine olanak tanımaktadır. Bu konteynerleştirme stratejisi, geliştirme ortamının tüm paydaşlar için izole, tutarlı ve tekrarlanabilir bir yapıda kurulmasını garanti altına alarak "benim makinemde çalışıyor" sorununu teknik olarak ortadan kaldırmaktadır.

Sistemin farklı yaşam döngüsü aşamaları için özelleşmiş yapılandırma dosyaları mevcuttur; bu doğrultuda üretim (production) ortamı gereksinimleri ve mikroservis mimarisi varyantları, operasyonel karmaşıklığı yönetmek adına bağımsız dosyalarda tanımlanmıştır. Bu modüler Docker mimarisi, servisler arası ağ (networking) yapılandırmalarını ve hacim (volume) yönetimini merkezileştirirken, uygulamanın bulut tabanlı modern dağıtım (deployment) süreçlerine tam uyum sağlamasına zemin hazırlamaktadır.

4.10.3. Ortam değişkenleri ve üretim güvenliği

Sistem mimarisinde yer alan örnek kimlik doğrulama parolaları ve JWT (JSON Web Token) imza anahtarları, yalnızca yerel geliştirme ve test süreçlerinde işlevsellik kontrolü sağlamak amacıyla kurgulanmıştır. Uygulamanın üretim (production) ortamına aktarılması aşamasında, verilerin gizliliğini ve bütünlüğünü korumak adına yüksek entropili, karmaşık kriptografik anahtarların kullanımı ve bu hassas verilerin "Secret Management" veya kasa yazılımı (HashiCorp Vault, AWS Secrets Manager vb.) çözümleriyle yönetilmesi teknik bir zorunluluktur.

Güvenlik yüzeyini daraltmak amacıyla, geliştirme aşamasında esnek tutulan CORS (Cross-Origin Resource Sharing) erişim listeleri, canlı ortamda yalnızca kurumun yetkili alan adlarıyla (domain) sınırlandırılarak Cross-Site Request Forgery (CSRF) ve benzeri saldırı vektörlerine karşı koruma sağlanmalıdır. Benzer şekilde; Seq günlükleme arayüzü, MinIO yönetim paneli ve veritabanı yönetim araçları gibi kritik idari arayüzlerin dış dünyaya açık olması engellenmeli; bu servislere erişim yalnızca güvenli VPN tünelleri veya IP kısıtlamalı iç ağlar üzerinden gerçekleştirilmelidir. Bu savunma stratejisi,

platformun operasyonel güvenliğini maksimize ederken, kritik altyapı bileşenlerinin yetkisiz erişimlere karşı izole edilmesini garanti altına almaktadır.

4.11. REST ve örnek sözleşme

Sistem mimarisinde uygulama programlama arayüzü (API) kaynak adresleri, RESTful tasarım prensipleri doğrultusunda çoğul isim tamlamalarıyla yapılandırılmıştır. İsteklerin işlenmesi ve yanıtlanması süreçlerinde; istemci kaynaklı hatalar (400 Bad Request), yetkilendirme sorunları (401 Unauthorized), kaynağın bulunamaması (404 Not Found) ve başarılı işlem sonuçları (201 Created) gibi durumlar, ilgili standart HTTP durum kodları ile birbirinden kesin hatlarla ayrıştırılmaktadır.

Sipariş oluşturma (Order Placement) süreci kapsamında istemci tarafından gönderilen veri paketinde; ürün listesi, miktar bilgileri ve detaylı teslimat adresi verileri JSON formatında beklenmektedir. Sunucu tarafında gerçekleştirilen doğrulama ve kayıt operasyonlarının ardından, başarılı yanıt gövdesinde (response body) sistem tarafından üretilen sipariş nesnesi ve işlem detayları istemciye iletilmektedir. Platformun ölçeklenebilirliği ve geliştirici deneyimini (Developer Experience) artırmak amacıyla, tüm uç noktaların OpenAPI (Swagger) spesifikasyonları ile şematize edilmesi ve dokümanite edilmesi, sistemin bakım maliyetlerini düşürecek ve entegrasyon süreçlerini standartlaştıracak stratejik bir gereklilik olarak öngörülmektedir.

4.12. Sürüm kontrol

Yazılım geliştirme yaşam döngüsü boyunca kaynak kod yönetimi, sürüm kontrol sistemi olan Git aracılığıyla titizlikle takip edilmiştir. Projenin evrimsel süreci; yeni özelliklerin geliştirilmesi, hata ayıklama operasyonları ve sistem iyileştirmeleri için oluşturulan bağımsız "özellik dalları" (feature branches) üzerinden yürütülmüştür. Bu metodoloji, ana kod hattının (main branch) her zaman kararlı ve yayına hazır kalmasını sağlarken, eş zamanlı geliştirme faaliyetlerinin birbirini olumsuz etkilemesini engelleyen izole bir çalışma ortamı sunmuştur.

Geliştirme sürecindeki bu yapılandırılmış yaklaşım, projenin teknik olgunluk düzeyini ve iş birliği standartlarını yansıtmaktadır. Dallanma stratejileri, kod inceleme (code review) süreçleri ve ana dallara yapılan birleştirme (merge/pull request) geçmişine dair operasyonel özetler, sistemin gelişim safhalarını somut bir şekilde ortaya koymak amacıyla tez ekinde detaylandırılmıştır. Bu dökümantasyon, yazılımın yalnızca nihai

halini değil, aynı zamanda mühendislik disiplini içerisinde katedilen aşamalı ilerlemeyi de kayıt altına almaktadır.

4.13. Bölüm özeti

Bu bölümde, Fuiira E-Commerce platformunun uygulama safhası; tercih edilen teknoloji yığını, veri modelleme stratejileri, merkezi storebackend mimarisi, üç farklı kullanıcı arayüzü ve sistemin temel ticari iş akışları çerçevesinde kapsamlı bir biçimde özetlenmiştir. Mevcut çalışma, teorik gereksinimlerin modern yazılım mühendisliği prensipleriyle nasıl somut bir elektronik ticaret çözümüne dönüştürüldüğünü teknik detaylarıyla ortaya koymuştur. Bir sonraki bölümde, elde edilen bulgular ışığında çalışmanın genel sonuçları değerlendirilecek ve platformun ölçeklenebilirliğini artırmaya yönelik ileriye dönük geliştirme önerileri sunulacaktır.

5. SONUÇLAR VE ÖNERİLER

5.1. Sonuçlar

Gerçekleştirilen çalışma kapsamında, çok satıcılı (multi-vendor) e-ticaret ekosistemi, modüler bir yaklaşımla üç bağımsız web arayüzü ve merkezi bir REST API çatısı altında konsolide edilmiştir. Bu mimari bütünlük, farklı kullanıcı rollerinin

(müşteri, satıcı, yönetici) özelleşmiş ihtiyaçlarını karşılarken, veri tutarlılığını ve operasyonel verimliliği tek bir merkezden yönetme imkanı sunmuştur.

Ön yüz geliştirme süreçlerinde React ve TypeScript ikilisinin Vite araç seti ile birlikte kullanımı; bileşen tabanlı geliştirme hızı, tip güvenliği (type safety) ve modern paketleme optimizasyonları sayesinde projenin sürdürülebilirliğini ve bakım kolaylığını maksimize etmiştir. Arka uç katmanında ise Express çerçevesinin esnekliği ile Prisma ORM'in sağladığı şema odaklı veri yönetimi, veritabanı yapısının uygulama koduyla senkronize bir şekilde evrilmesine olanak tanımıştır. Bu teknoloji yığını birlikteliği, hem tip güvenli sorgular üretilmesine hem de karmaşık veri ilişkilerinin (ilişkisel tablolar, işlemler ve kısıtlamalar) hatasız bir şekilde muhafaza edilmesine doğrudan katkı sağlamıştır.

5.2. Karşılaşılabilecek zorluklar

Eş zamanlılık (concurrency) yönetimi, özellikle stok gibi sınırlı kaynakların tüketildiği senaryolarda veri bütünlüğünün korunması için kritik bir teknik eşittir. Aynı ürün kalemine gelen yoğun ve eş zamanlı taleplerde "stok yarışı" (race condition) oluşmaması adına, veritabanı seviyesinde iyimser kilitleme (optimistic locking) veya yüksek trafikli anlarda Redis tabanlı dağıtık kilit (distributed lock) mekanizmalarının kullanımı, tutarlılığı garanti altına alan temel stratejilerdir.

Sistemin çalışma döngüsünde Redis (önbellek), ilişkisel veritabanı (kalıcı veri) ve MinIO (nesne depolama) gibi farklı katmanların eş güdümlü olarak yönetilmesi, yüksek düzeyde operasyonel yetkinlik ve orkestrasyon becerisi gerektirmektedir. Bu bileşenlerin Docker ve benzeri konteyner teknolojileriyle standartlaştırılması, dağıtım (deployment) süreçlerindeki karmaşıklığı azaltsa da, her bir servisin kaynak tüketimi ve sağlık durumunun izlenmesi süreklilik arz eden bir görevdir.

Platformdaki veri hacmi ve kullanıcı etkileşimi arttıkça, liste döndüren uç noktaların (endpoints) performansını korumak amacıyla stratejik bir indeksleme ve önbellek planı devreye alınmalıdır. Sık kullanılan sorgu kriterlerine göre optimize edilen veritabanı indeksleri ve statikleşen veri setlerinin Redis üzerinde konumlandırılması, hem sunucu yanıt sürelerini (TTFB) düşürmekte hem de altyapı maliyetlerini optimize ederek ölçeklenebilir bir büyüme zemini hazırlamaktadır.

5.3. Kısıtlılıklar

Mevcut mimari kurgu, tek bir monolitik API (monolithic API) yapısı etrafında şekillenmiştir; bu durum geliştirme ve dağıtım süreçlerinde başlangıç aşaması için operasyonel kolaylık sağlamaktadır. Ancak, services klasörü altında yapılandırılan iş mantığı süreçleri, tüm sistem özellikleri için her zaman birebir paralel veya tam yalıtılmış (isolated) şekilde çalışmayabilir. Bu tür bir yapı, sistem ölçeklendikçe servisler arası sıkı bağımlılıklara (tight coupling) yol açabileceğinden, gelecekteki mikroservis dönüşüm potansiyeli bu mimari kısıtlar göz önünde bulundurularak değerlendirilmelidir.

Yazılım yaşam döngüsünde test otomasyonunun belirli bir kapsamda sınırlı kalması, projenin hızla prototiplenmesine olanak tanısa da, bu durumun ilerleyen süreçlerde "teknik borç" (technical debt) olarak kayda geçirilmesi kritik bir yönetim hamlesidir. Özellikle karmaşık sipariş döngüleri ve yetkilendirme katmanları için birim ve entegrasyon testlerinin eksikliği, sistem genişledikçe regresyon hatalarının riskini artıracaktır.

Ticari ve hukuki düzlemde ise, ödeme sistemleri entegrasyonu ve kişisel verilerin korunması (KVKK/GDPR) gibi hassas konular, projenin bir "canlı ürün" (production-ready product) haline gelmesi aşamasında yalnızca teknik değil, hukuk ve regülasyonlar çerçevesinde de ele alınmalıdır. Veri saklama politikaları, rıza metinleri ve ödeme sağlayıcılarının güvenlik standartları (PCI-DSS), sistemin yasal uyumluluğunu ve son kullanıcı güvenini tesis etmek adına mimarinin ayrılmaz bir parçası olarak yapılandırılmalıdır.

5. 4. Gelecek çalışmalar

Sistemin üretim (production) ortamına geçişi ve profesyonel bir e-ticaret operasyonuna dönüşümü için belirlenen yol haritası, mimariyi daha dayanıklı ve ölçeklenebilir kılacak kritik geliştirme dikeyinden oluşmaktadır. Mevcut "ödendi" varsayımı yerine, gerçek bir ödeme hizmet sağlayıcısı (iyzico, Stripe vb.) entegrasyonu ile birlikte ödeme durumlarını asenkron olarak doğrulayan Webhook mekanizmaları kurgulanacaktır. Sipariş onayları, kargo güncellemeleri ve güvenlik bildirimleri gibi kullanıcı etkileşimleri için e-posta ve SMS servisleri sisteme dahil edilecek; bu yoğun operasyonel yükü ana API döngüsünden ayırmak amacıyla RabbitMQ tabanlı asenkron bir bildirim mimarisi oluşturulacaktır.

Ürün keşif performansını artırmak amacıyla Elasticsearch veya Meilisearch gibi özelleşmiş arama motorlarının entegrasyonu ve monolitik yapının mikroservislere bölünmesi aşamasında tüm trafiği yönetecek bir API Gateway katmanının devreye alınması planlanmaktadır. Mevcut REST API sözleşmesi kullanılarak deneyimin mobil cihazlara taşınması, yerel (Native) veya hibrit uygulamaların geliştirilmesiyle sağlanacaktır.

Süreçlerin sürdürülebilirliği için GitHub Actions gibi araçlarla CI/CD hatları kurulacak, kodun her gönderiminde otomatik testlerden geçerek yayına alınması standartlaştırılacaktır. Üretim ortamındaki hataların, performans darboğazlarının ve kaynak tüketiminin anlık takibi için Prometheus, Grafana veya Sentry gibi gelişmiş izleme (monitoring) ve günlükleme altyapıları sisteme eklenecektir. Bu teknik geliştirmeler, projenin yüksek trafikli ve ticari risk taşıyan profesyonel bir ekosisteme dönüşmesi için gerekli olan teknolojik temel taşlarını temsil etmektedir.

5. 5. Etik ve yasal hususlar

Sistem mimarisinin canlı bir ticari platforma dönüşmesi aşamasında, teknik yeterlilik kadar hukuki uyumluluk da öncelikli hale gelmektedir. Kişisel verilerin işlenmesi süreçleri, KVKK (Kişisel Verilerin Korunması Kanunu) ve uluslararası standartlar çerçevesinde yapılandırılmalıdır. Bu kapsamda, kullanıcıların üyelik bilgileri, adres verileri ve alışveriş geçmişi gibi hassas verilerin saklanma süreleri, işleme amaçları ve anonimleştirme stratejileri veri envanterinde açıkça tanımlanmalıdır.

Finansal güvenlik açısından, kredi kartı ve banka kartı verileri hiçbir aşamada platformun kendi veritabanında düz metin veya şifrelenmiş olarak saklanmamalıdır. Bunun yerine, PCI-DSS uyumluluğuna sahip yetkili ödeme sağlayıcıları (Payment Service Providers) aracılığıyla sunulan tokenization (belirteçleştirme) teknolojisi kullanılmalıdır. Bu yöntemle, hassas kart bilgileri sağlayıcı tarafında güvenli bir şekilde saklanırken, platform tarafında yalnızca bu veriyi temsil eden ve başka bir amaçla kullanılamayan benzersiz bir "token" tutulmaktadır.

Platformun hukuki zeminini sağlamlaştırmak amacıyla; kullanıcıların haklarını belirleyen Kullanım Koşulları (Terms of Service), veri işleme süreçlerini detaylandıran Aydınlatma Metni ve tarayıcı tabanlı veri takibi için Çerez Bilgilendirmesi (Cookie Policy) arayüz katmanında erişilebilir ve onaylanabilir şekilde sunulmalıdır. Bu

dökümantasyon ve teknik önlemler, sistemin yasal sorumluluklarını yerine getirmesini sağlarken, son kullanıcı nezdinde kurumsal bir güven ve şeffaflık tesis etmektedir.

5. 6. Sonuç

Fuira E-Commerce, modern web teknolojileri ve yazılım mimarisi prensiplerini temel alarak, çok paydaşlı bir pazar yeri ekosisteminin teknik olarak nasıl inşa edilebileceğini somut bir model üzerinden ortaya koymaktadır. Geliştirilen bu yapı; kimlik doğrulama stratejilerinden atomik veritabanı işlemlerine, veri izolasyonundan asenkron süreç yönetimine kadar geniş bir mühendislik spektrumunu kapsayarak hem yazılım geliştirme pratikleri hem de akademik öğrenme süreçleri için nitelikli bir referans noktası sunmaktadır.

Mevcut teknoloji yığını ve modüler kod organizasyonu, sistemin genişletilebilirliğini ve bakımını kolaylaştıracak şekilde kurgulanmıştır. Platformun prototip aşamasından profesyonel bir ticari ürüne dönüşebilmesi için; gerçek ödeme geçidi entegrasyonları, KVKK uyumluluğu, gelişmiş mesaj kuyruğu yapıları ve kapsamlı test otomasyonları gibi stratejik iyileştirme adımlarının atılması gerekmektedir. Bu önerilen teknolojik ve hukuki güncellemeler hayata geçirildiğinde, sistem yüksek trafikli ve güvenli bir ticari operasyonu yönetecek olgunluğa erişmiş olacaktır.

5.2 Öneriler

Gelecek Dönem Çalışmaları ve Geliştirme Önerileri

Fuira E-Commerce platformunun mevcut prototip aşamasından profesyonel bir ticari ürüne evrilmesi için odaklanılması gereken temel alanlar şunlardır. Mevcut sembolik ödeme onayının yerine iyzico veya Stripe gibi gerçek bir ödeme geçidi entegrasyonu yapılmalıdır. Kart verileri asla yerel veritabanında tutulmamalı, PCI-DSS uyumlu tokenization yöntemleri ve işlem doğruluğu için Webhook yapıları kullanılmalıdır. Sipariş sonrası e-posta gönderimi veya ağır veri işleme süreçleri ana API döngüsünden çıkarılmalıdır. RabbitMQ gibi mesaj kuyrukları kullanılarak bu işlemlerin arka planda yürütülmesi, sistemin genel tepki süresini ve kullanıcı deneyimini iyileştirecektir. Veri hacmi arttıkça oluşacak darboğazları önlemek adına Redis üzerinde agresif bir önbellekleme stratejisi izlenmeli ve ürün keşif hızını artırmak için Elasticsearch gibi metin tabanlı arama motorları mimariye dahil edilmelidir. Kişisel verilerin işlenmesi, saklanması ve imhası süreçleri KVKK regülasyonlarına tam uyumlu

hale getirilmelidir. Kullanıcı aydınlatma metinleri ve veri gizliliği protokolleri teknik altyapı ile desteklenmelidir. Teknik borcu minimize etmek için kritik sipariş akışlarını kapsayan entegrasyon testleri artırılmalı ve GitHub Actions gibi araçlarla kodun her gönderiminde otomatik test edilip yayına alındığı bir sürekli dağıtım hattı kurulmalıdır. Bu adımlar atıldığında, Fuir E-Commerce bir bitirme projesi olmanın ötesine geçerek, gerçek dünya trafiğini ve ticari risklerini yönetebilecek bir olgunluğa erişmiş olacaktır.

6. KAYNAKLAR

- Camara, C., Basso, F., & dos Santos, A. L. (2021). Micro-frontends: A systematic mapping study. *Journal of Systems and Software*, 176, 110935. <https://doi.org/10.1016/j.jss.2021.110935>
- Dragoni, N., Lanese, I., Larsen, S. T., Mazzara, M., Mustafin, R., & Safina, L. (2017). *Microservices: Yesterday, today, and tomorrow*. Springer. https://doi.org/10.1007/978-3-319-67425-4_12
- Grolinger, K., Higashino, W. A., Tiwari, A., & Capretz, M. A. (2013). Data management in cloud environments: NoSQL and NewSQL data stores. *Journal of Cloud Computing*, 2(1), 22. <https://doi.org/10.1186/2192-113X-2-22>
- Han, J., Haihong, E., Le, G., & Du, J. (2011). Survey on NoSQL database. 6th International Conference on Pervasive Computing and Applications. <https://doi.org/10.1109/ICPCA.2011.6106531>
- Kleppmann, M., Kreps, J., & Wang, J. (2018). Data consistency in distributed systems: A survey. *ACM Computing Surveys*, 51(1). <https://doi.org/10.1145/3184594>
- Pahl, C. (2015). Microservices: A systematic mapping study. CLOSER 2015. <https://www.scitepress.org/PublishedPapers/2016/57855/57855.pdf>
- Pautasso, C., Zimmermann, O., & Leymann, F. (2008). Restful web services vs. big web services. *WWW*, 2008. <https://doi.org/10.1145/1367497.1367606>
- Taibi, D., Lenarduzzi, V., & Pahl, C. (2018). Processes, motivations, and issues for migrating to microservices architectures. *IEEE Cloud Computing*, 5(5), 22–32. <https://doi.org/10.1109/MCC.2017.4250931>
- Villamizar, M., Garcés, O., Ochoa, L., et al. (2016). Infrastructure cost comparison of microservices and monolithic architectures. *CCGrid* 2016. <https://doi.org/10.1109/CCGrid.2016.37>
- Waseem, M., Liang, P., Shahin, M., Di Salle, A., & Márquez, G. (2021). Design, monitoring, and testing of microservices systems: The practitioners' perspective. *Journal of Systems and Software*, 182, 111061. <https://doi.org/10.1016/j.jss.2021.111061>
- Kanthed, S. (2023). Redis vs. Memcached in microservices architectures: Caching strategies. *International Journal of Multidisciplinary Research and Growth Evaluation*, 4(3), 1084–1091. <https://doi.org/10.54660/IJMRGE.2023.4.3.1084-1091>

- Kaynak: Bernstein, D. (2014). Containers and Cloud: From LXC to Docker to Kubernetes. IEEE Cloud Computing, 1(3), 81–84.
<https://doi.org/10.1109/MCC.2014.51>
- Kaynak: Li, W., Lemieux, Y., Gao, J., Zhao, Z., & Han, Y. (2019). Service Mesh: Challenges, State of the Art, and Future Prospects. 2019 IEEE International Conference on Service-Oriented System Engineering (SOSE), 94–102.
<https://doi.org/10.1109/SOSE.2019.00026>
- Kaynak: Humble, J., & Farley, D. (2010). Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation. Addison-Wesley Professional.
<https://database-systems.org/resources/continuous-delivery.pdf>
- Kaynak: Bernstein, D. (2014). Containers and Cloud: From LXC to Docker to Kubernetes. IEEE Cloud Computing, 1(3), 81–84.
<https://doi.org/10.1109/MCC.2014.51>
- Kaynak: Li, W., Lemieux, Y., Gao, J., Zhao, Z., & Han, Y. (2019). Service Mesh: Challenges, State of the Art, and Future Prospects. 2019 IEEE International Conference on Service-Oriented System Engineering (SOSE), 94–102.
<https://doi.org/10.1109/SOSE.2019.00026>
- Kaynak: Chen, L. (2015). Continuous Delivery: Huge Benefits, but Challenges Too. IEEE Software, 32(2), 50–54.
<https://doi.org/10.1109/MS.2015.27>
- Kaynak: Castro, P., Ishakian, V., Muthusamy, V., & Slominski, A. (2019). Serverless Computing: Current Trends and Open Problems. ResearchGate Publishing.
<https://doi.org/10.48550/arXiv.1710.05873>
- Kaynak: Wambler, J., Rosendo, D., & Cappiello, C. (2018). Observability in Microservices-based Systems: A Mapping Study. 2018 IEEE/ACM International Conference on Software and System Processes (ICSSP).
<https://doi.org/10.1145/3202710.3203154>

EKLER

EK-1 Uygun bir başlık buraya yazılmalıdır. Ekler tez şablonuna uygun olacak şekilde düzenlenmelidir.